Peekaboo: What Right / Wrong Feedback Buys a Small Decision Model, Measured Against the Model's Own Answer-Key Recipe

Abhijith Neil Abraham Fariz Rahman Fadil Rahman

October 2026

Abstract

Vega and Laya are small models that answer typed questions (pick one of these options, yes or no, a rating) with calibrated probabilities, and each ships with a recipe for improving on a user's task from labelled rows. A deployed model rarely gets labels; it gets verdicts on its own answers. We measure how much of each vendor's labelled-data recipe right / wrong feedback recovers, on the benchmarks the vendors use themselves, with the controls that question needs: the same add-on trained on the labels, with and without early stopping; a label prior with no input features trained from the same verdicts; the released model with the same feedback-fitted temperature; the vendor's own method under both of its selection rules; and, for Laya, the matched in-domain fine-tune with the encoder unfrozen. Every feedback arm is run with eight passes over its rows, the protocol of our first version, and with one pass in which each row is rated once, which is what a deployment sees; we also run the released model's own answers, best or sampled, logged and rated once, and feedback that is flipped, missing or skewed to complaints. Intervals resample training seeds and row clusters and are Holm-adjusted within families. Under eight passes a REINFORCE add-on with a KL term to the released model lands within about two points of the same adapter trained on labels on Vega (+11.3 against +13.2 on DecisionBench at 0.8B), largely because eight on-policy verdicts reveal the gold answer for most rows; the expected update is cross-entropy weighted by the model's own probability of the gold answer, and which rows get fixed follows that weighting. One on-policy pass keeps about half to two thirds of the gain (+6.7), but most of the shortfall is optimisation: a single verdict per row on the released model's sampled answers, logged once and trained off-policy to convergence, gains +8.3 on DecisionBench and is level with eight on-policy passes on typed-decisions and emotion. It still trails the labels. On Laya with the encoder unfrozen, the matched labelled fine-tune beats eight passes of verdicts by 6.5 points on typed-decisions and 7.9 on emotion. Against VegaML's TTT head under its current rule, the feedback-trained adapter is well ahead only on DecisionBench and well behind on emotion. The temperature refit from verdicts is top-label temperature scaling, and much of its ECE advantage over the released model is available to the released model itself. We also disclose two public-menu tasks and two DecisionBench tasks whose gold answers are learnable from the option list, a shipped Vega adapter trained on our typed-decisions feedback pool, and the class-balanced pools that distorted the first version's emotion results. Code, records and the review of the first version are released.

1 Introduction

Vega [37] and Laya [6] are small models built for one job: given a situation and a typed question (pick one of these options, yes or no, a rating), return a probability for each option in one forward pass, fast enough and calibrated enough to sit inside an agent loop and route the uncertain cases to a person. Both ship with a recipe for getting better at a user's task, and both recipes want labelled rows. Vega's is a per-task classifier fitted to its frozen features (test-time training, TTT) or an adapter trained the same way; Laya's is a whole-

model fine-tune, and its released `laya-typed-decisions` checkpoint is that recipe applied to its headline benchmark.

A deployed model rarely gets labels. What it gets, as a by-product of being used, is a verdict on its own answers: the suggested tool call was cancelled, the flagged message was fine after all, the escalated ticket did need escalating. Each event says whether the answer was right, not which answer was right. Learning a classifier from that signal is the bandit multiclass problem [7, 9, 26], and improving a deployed NLP system from binary feedback simulated from gold labels is the protocol of Gao, Choi and Artzi [14]. None of that is new, and this paper does not claim a new learning setting or a new estimator.

What we wanted to know is narrower and, we think, still worth measuring: for these two models, on the benchmarks their own authors use, how much of the labelled-data recipe does right / wrong feedback recover, under what protocol, and what are the gains made of? The first version of this paper [1] answered with a grid in which every feedback method made eight passes over its rows, drawing a fresh answer and a fresh verdict each time, and it read the result as "one bit per row is nearly as good as the label". A review of that version[2] showed that this reading does not follow. Eight on-policy verdicts identify the gold answer for most rows, the expected update of REINFORCE with a 0/1 reward is cross-entropy reweighted by the model's own probability of the gold answer [10, 58], the switch that was meant to catch regressions compared the wrong quantity, the Laya comparison set an in-domain model against an out-of-domain one, and several of the gains were partly a label prior or a benchmark artifact. This version is the measurement done again with those things fixed, and with the questions a deployment actually raises added.

The method, Peekaboo, is unchanged in its parts: an add-on next to the untouched released model (a rank-8 gated LoRA adapter on Vega's decision engine; a copy of Laya's decision head, or of its head and encoder), trained by REINFORCE on the verdict of a sampled answer with a running baseline and a KL term towards the released probabilities; a single temperature refitted from verdicts alone; and a switch that keeps the add-on only where held-out verdicts say it beats the released model. What is new in this version is the accounting:

- every feedback arm is run both with eight passes (the information-rich protocol) and with one pass in which each row is rated exactly once, which is the number a deployment should expect; the same for the answer-key arms, at matched epochs;
- feedback as a deployment delivers it: the released model's own best answers rated once and learned off-policy, sampled answers logged once, ratings that are missing, flipped, or skewed towards complaints;
- the controls a reviewer will ask for: the same add-on trained on the labels, with and without early stopping; a label-prior baseline with no input features, trained from the same verdicts; the released model with the same feedback-fitted temperature; TTT under both of VegaML's selection rules; for Laya, the matched in-domain answer-key fine-tune with the encoder unfrozen;
- intervals that resample training seeds as well as rows, clustered by task or case, with Holm adjustment within each family of comparisons and a non-inferiority test at a stated margin; proper scores next to ECE;
- disclosure of what the benchmarks contain: a TruthfulQA task whose gold answer is always first, a DecisionBench task whose gold is readable from the option structure, and a shipped Vega adapter that was fitted with labels on the very rows that serve as its typed-decisions feedback pool.

What we found. Under eight passes, verdicts come close to labels on Vega: the feedback-trained adapter is 1.9 points below the same adapter trained on labels and early-stopped on DecisionBench at 0.8B and level at 4B (+0.6), within about two points on the public menu, typed-decisions and AG News, and further behind on emotion (−4.9, −3.5). This is less remarkable than it sounds: on DecisionBench each training row produced 4.88 verdicts of "right" on average, each of which names the gold answer, so the two arms were mostly

[1] https://github.com/abhijithneilabraham/peekaboo, commit `cd52a9f`.

[2] The review is in the repository under `reports/critique.zip`; Appendix A lists what it found and what changed.

training on the same labels with different weights. With one on-policy pass the adapter keeps about half to two thirds of the gain (+6.7 and +3.2 on DecisionBench against +11.3 and +5.7). The logged arms show that most of that loss is optimisation rather than information: one verdict per row on the released model's sampled answers, trained off-policy for eight passes, gains +8.3 and +4.3 on DecisionBench and is level with eight on-policy passes on typed-decisions and emotion. Logging the released model's best answers instead of samples gains a little less. Flipped, missing or complaint-skewed verdicts cost a point or two. Against VegaML's TTT head under its current selection rule, the feedback-trained adapter is well ahead on DecisionBench, a point ahead on AG News, level on the public menu and typed-decisions, and well behind on emotion. On typed-decisions and the public menu, a per-task label prior with no input features recovers most of the gain from the same verdicts; on DecisionBench and emotion it does not. On Laya, the matched labelled fine-tune with the encoder unfrozen beats the verdict-trained one by 6.5 points on typed-decisions and 7.9 on emotion, and is level elsewhere; the first version's "beats Laya's own fine-tune" compared in-domain training with a transfer model. The temperature refit is necessary for the probabilities and is top-label temperature scaling; the switch, now comparing best answers, returns the released model where the add-on hurts. With Vega's shipped typed-decisions adapter switched off, so that the released model starts weak on that benchmark, labels beat verdicts by 18.6 points at 0.8B, the clearest picture of what verdicts on sampled answers cannot do when the model rarely samples the right one.

2 Related work

Learning a classifier from right / wrong. Multiclass learning in which the learner is told only whether its prediction was correct goes back to the Banditron [26]; Crammer and Gentile [7] evaluated it on text classification, and Daniely and Halbertal [9] bound the price of bandit information at $O(k \log k)$ more mistakes in the realisable case, a more useful figure than the "$\log_2 k$ bits against one" heuristic our first version used. Wang and Qiao [51] train a neural classifier online from correctness feedback with an importance-weighted cross-entropy and derive conformal sets from the same feedback. Learning a structured predictor from the reward of its own output is bandit structured prediction [28, 38, 47], shown to work with real human ratings [30] and, at eBay, with implicit feedback where explicit ratings were too noisy [29]; the off-policy branch learns from logged feedback with importance weights [12, 24, 33, 50].

Improving a deployed system from its users. The protocol closest to ours is Gao, Choi and Artzi [14]: binary rewards simulated from gold labels, on-policy policy-gradient learning from the model's own answers, supervised training as the reference line, and the finding that the initial model's quality decides whether learning works. Its follow-up collected real feedback over nine deployment rounds [15]. Campos et al. [3] and Hancock et al. [20] improve deployed QA and dialogue systems from binary signals. Our contribution relative to this line is the object of study, two released typed-decision models with calibrated probabilities, the comparison against their vendors' own labelled-data recipes, and the matched one-pass and logged-feedback arms.

What a 0/1 reward does to the update. With a single gold answer y and reward $r = \mathbf{1}[a = y]$, the expected REINFORCE update is $\mathbb{E}_{a\sim\pi}[(r-b)\nabla \log \pi(a)] = \pi(y)\,\nabla \log \pi(y) = \nabla \pi(y)$: cross-entropy on the gold answer, reweighted by the policy's own probability of it. This is the mean-absolute-error objective of the label-noise literature [16, 58], known to under-fit rows the model already gets badly wrong, and the identity Davis and Recht [10] give for binary-reward RL in general. Filtered self-training [11, 54, 56] is the same mechanism without the negative term: every correct sample is a supervised step on the revealed label. Yue et al. [55] show that RL with verifiable rewards mostly re-ranks what the base model could already produce; Choshen et al. [4] and Kiegeland and Kreutzer [27] make the same point for bandit NMT and attribute much

of the gain to the sharpening of the output distribution. Our fix / break analysis (Section 6.3) is a direct check of these predictions on our models.

KL-regularised fine-tuning and forgetting. A KL penalty to a reference policy is standard in RLHF [5, 39, 49, 62] and offline RL for dialogue [23]; DPO [42] and KTO [13] fold it into a supervised objective, KTO from binary desirable / undesirable labels, and REINFORCE with a baseline remains competitive with more elaborate estimators [1]. Shenfeld et al. [46] show that on-policy RL forgets less than supervised fine-tuning because it stays closer to the base distribution, which bears on how Laya's whole-model fine-tune and our encoder-level add-on behave away from their training benchmark.

Few-shot heads against a zero-shot model. A linear probe fitted on a few labelled rows can be worse than the zero-shot model it replaces; CLIP needed about four shots per class to match its own zero-shot accuracy [41], and anchoring the fine-tuned model to the zero-shot one is the standard remedy [32, 53]. VegaML's TTT head is such a probe, and the rule that decides when it replaces the engine changed in vegaml 0.12.3 to require beating the engine as well as chance; we report both rules.

Label priors and label shift. Part of what any adapter learns from a benchmark's rows is its label distribution and answer-position bias. Contextual calibration [59], batch calibration [61], prior adjustment [34, 44] and the choices-only baselines of multiple-choice evaluation [2, 60] all measure this; our label-prior baseline (Section 6.3) is the feedback-trained version.

Calibration after policy gradient. The expected 0/1 reward of a sampled answer is the linear score, which is improper [17]: it is maximised by putting all the mass on the argmax, and under a KL term of weight β the optimum raises the log-odds of a reliably right answer by up to $1/\beta$ nats. Over-confidence after RL fine-tuning is documented for RLHF policies and fixed with a single temperature [25], and binary-correctness RL hurts calibration unless a proper score is added to the reward [8]; Laya's own RLCD uses proper scoring rules for this reason. Fitting a temperature from correctness alone is top-label calibration [19]: ECE itself [18, 36] consumes nothing but (confidence, right / wrong) pairs. Our "honesty fix" is that, written for sampled rather than argmax answers; we no longer present it as calibration without labels in any new sense. ECE is a biased, non-proper estimate [31, 43], so we report NLL, Brier and AURC beside it.

3 The two decision models, and what they already know

Vega. Vega [37] reads the situation, question and options with a frozen Qwen3.5 backbone [40] of 0.8B or 4B parameters, pools hidden states from two intermediate layers, and hands them to a small decision engine (14.3M parameters at 0.8B, 30.9M at 4B) that the authors call a physics engine: each option is an energy well in a 64-dimensional landscape, a particle is released, and $P(a) = \mathrm{softmax}(-E_a/\tau(s))$ with a per-question temperature computed from the particle's trajectory. The backbone was never trained; the engine and two gated adapters were, on an undisclosed corpus. The two adapters matter here. `public_intents` was trained on six intent datasets, including banking77 and clinc150, which are also public-menu tasks. `typed_decisions` was fitted with labels on the training split of the public typed-decisions dataset, and its gate opens on every one of the 8,000 typed-decisions rows we use. That training split is exactly the pool our feedback methods learn from on typed-decisions. So on that benchmark "base Vega" is a model that has already seen the labels of its feedback rows, and it is 5 to 6 points more accurate on the pool than on the test split; we report typed-decisions on Vega both as released and with the shipped adapters switched off (Section 6.7). Vega and Laya come from the same developer, and we pin vegaml 0.8.0, the version the README's benchmark notebook uses.

Laya. Laya [6] is a ModernBERT-large encoder [52] (395M parameters) whose input carries the situation, the question and every option, each option introduced by a marker token, followed by a two-layer transformer head (26.5M parameters) that scores the marker positions; the scores are divided by a shipped temperature per (question type, option-count bucket). It was trained with RLCD on undisclosed data and reads 512 tokens, so rows whose option markers do not fit are unanswerable for every method built on it (all of clinc150_oos; 49 DecisionBench test rows). `laya-typed-decisions` is the developer's whole-model fine-tune on typed-decisions' 6,000 training decisions, with a 1,024-token window. On the five other benchmarks it is a transfer model that never saw a row, which is how we present it.

4 Peekaboo

4.1 Setting, and what "one bit" means under each protocol

A decision model is a policy $\pi_0(a \mid s)$ over the options a of a typed question s. In deployment it commits to an answer and later receives $r \in \{0, 1\}$. We train an add-on π from a pool of (question, answer, verdict) records and score it on held-out test rows. The reward is computed from the benchmark's gold answer, so it is never wrong or missing unless we make it so (Section 6.2). Two protocols appear throughout:

- **eight passes**: the policy answers every training row once per pass for eight passes, each time sampling a fresh answer from its current probabilities and receiving a fresh verdict. A row can thus be rated up to eight times on different answers. Whenever a verdict is 1 the gold answer has been revealed outright, and for two-option questions any verdict is the label. Section 6.3 measures how often this happens: for most rows, it does. This is the protocol of our first version and of the headline comparisons with labelled training, which also make eight passes.
- **one pass**: every training row is answered once and rated once. This is what a deployment sees, and it is the number we now report as Peekaboo's deployment figure. It also takes one eighth of the gradient steps, so we run the answer-key arms for one pass as well.

4.2 The add-on

The released weights are never changed. On Vega a gated LoRA adapter [22] of rank 8 is attached to seven of the engine's linear layers with VegaML's own `attach_adapter` (409k parameters at 0.8B, 857k at 4B, most of them in the gate network); its gate starts open and the shipped adapters keep routing as released. The policy is Vega's usual calibrated output computed from the adapted energies. On Laya we train a copy of the decision head with the encoder frozen (*head only*), or a copy of head and encoder (*encoder unfrozen*) at Laya's own encoder learning rate of 2.5×10^{-5}. The second is a fine-tune of the whole 421M model with a KL anchor, not a small add-on, and we describe it as such.

4.3 The update, and what it is in expectation

For a batch of questions the policy samples $a \sim \pi(\cdot \mid s)$, receives $r = \mathbf{1}[a \text{ is right}]$, and minimises

$$\mathcal{L} = -(r - b) \log \pi(a \mid s) + \beta \, \mathrm{KL}\big(\pi(\cdot \mid s) \,\|\, \pi_0(\cdot \mid s)\big), \qquad \beta = 0.1, \tag{1}$$

with b an exponential running mean of recent rewards ($b \leftarrow 0.9b + 0.1\bar{r}$, started at 0.5), Adam at 10^{-3} (Vega) or 10^{-4} (Laya head), batches of 32, gradient norm clipped at 1. Because the baseline is updated only after the batch, the expected gradient of the first term is $\pi(y \mid s) \nabla \log \pi(y \mid s)$, cross-entropy on the gold answer weighted by the policy's current probability of it. The answer-key arm minimises $-\log \pi(y \mid s)$ with the same KL term, rows, epochs and optimiser: the two differ only in that weight and in the sampling noise. That is the right frame for every "feedback against labels" number in this paper. The weighting predicts that

feedback will trail labels most on rows the base model ranks the gold answer low, and tie or lead where the gold answer is already near the top; Section 6.3 checks this. In our first version β was chosen in a pilot that trained and tested on DecisionBench's README rows, from $\{0, 0.1, 1\}$; it was not retuned, and the $\beta = 1$ runs of that grid are reported in Appendix A.

4.4 The honesty fix: top-label temperature scaling from verdicts

Policy-gradient training on an improper reward sharpens the probabilities (Section 6.4 gives the pre-fix ECE). We refit one temperature T on a held-out calibration split (20% of each task's feedback rows) by sampling $a \sim \pi$ at $T = 1$, observing r, and choosing

$$T^\star = \arg\max_T \sum_i \big[r_i \log p_T(a_i \mid s_i) + (1 - r_i) \log(1 - p_T(a_i \mid s_i))\big], \qquad p_T = \mathrm{softmax}(\log p / T), \quad (2)$$

over 61 log-spaced values in $[0.25, 32]$. Since the trained policy samples its argmax most of the time, this picks the same T as a fit on argmax correctness in nearly every run, i.e. it is top-label temperature scaling [19] from the verdicts a deployment already has. A temperature leaves the argmax unchanged, so accuracy is identical with and without it. The probabilities are floored at 10^{-12} before scaling, which ties the non-argmax options on rows where the trained policy had driven them below the floor; this affects NLL on those rows and is disclosed here because the first version did not.

4.5 The switch

On the calibration rows, the add-on's best answer and the released model's best answer are each rated (two verdicts per row), and the add-on is kept only if it was right more often. The first version rated one *sampled* answer from each; that statistic estimates the expected probability of the gold answer, which the honesty fix lowers for the add-on and over-confidence raises for the released model, so it threw away real argmax gains on calibration sets of 641 and 1,200 rows and kept a real loss on others. The sampled statistic is still computed and reported for comparison. The switch is a safeguard on the add-on as a whole, not a per-decision validator.

5 Experimental setup

5.1 Benchmarks and pools

Table 1 and Figure 1 describe the six benchmarks: the three in Vega's README (43 public tasks, JevBench [48], DecisionBench [21]), the one in Laya's README (typed-decisions [35]), and two classification sets we added for their large training splits (emotion [45], AG News [57]). Rows are read, asked and scored with VegaML's benchmark harness, vendored unmodified, and the test rows are identical to our first version's. Three things about the pools are disclosed here. The public menu's feedback pool is the 50 "fit" rows per task that VegaML's notebook gives its TTT head, drawn class by class, so it is class-balanced while the 60 test rows per task are not; the same loader gave our first version a class-balanced emotion pool against a naturally distributed test set, which cost every method trained on it on emotion. In this version the emotion and AG News pools are 4,000 rows drawn at random from the whole training split, so they carry the natural class mix a deployment's traffic would. The DecisionBench pool is the other rows of the README's 43 tasks (100 per task minus the 25-row test slice), drawn at random. On typed-decisions the pool is the public training split, which is also the training data of Vega's shipped adapter (Section 3). Two public tasks and two DecisionBench tasks have construction artifacts that an adapter can learn without reading the input: in `truthfulqa_mc1` the harness keeps the upstream option order, so the gold answer is option A in every row; in `pubmedqa` the gold is the majority class on 55% of test rows; in `moderation-civil-comments`

Table 1: The six benchmarks. "Feedback rows" are the rows whose verdicts a method may learn from (80%) and calibrate on (20%); no test row is ever used for learning.

benchmark	reported by	test rows	feedback rows	rows and pool
43 public tasks	Vega	2,535	2,134	43 public datasets as typed questions, up to 60 test rows each; feedback: VegaML's 50 fit rows per task, drawn class by class; macro-averaged over tasks
JevBench	Vega	231	(5-fold)	rules, policies and judgment calls in 18 families, paraphrase groups; no training split, every method cross-fits
DecisionBench	Vega	1,075	3,205	43 tasks in 28 domains; the README's 25-row slice per task is the test set, other rows of the same tasks the pool
typed-decisions	Laya	2,000	6,000	400 test and 1,200 training cases, five decisions each; the pool is the public training split, also the training data of Vega's shipped adapter
emotion	(ours)	1,503	4,000	six emotions of a tweet; pool: training rows drawn at random (natural class mix)
AG News	(ours)	1,912	4,000	four news sections; pool: training rows drawn at random

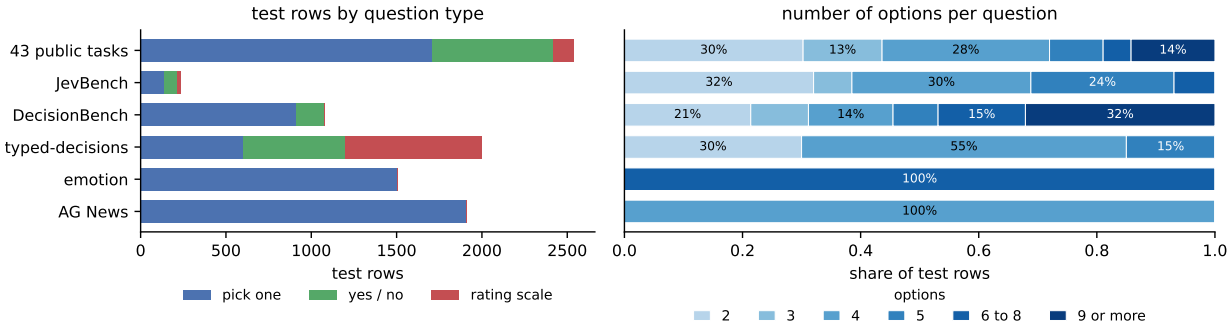


Figure 1: The six benchmarks' test rows, by question type (left) and by the number of options a question offers (right, as a share of that benchmark's rows).

the gold is the first option in 86% of rows; in `finqa-score` the options are always $\{x, 100x, x/100, 1.1x\}$ with gold x. We report every public and DecisionBench number with and without these tasks.

5.2 Protocol and statistics

Every method learns from its pool's feedback rows (or their labels) and is scored on the test rows. On JevBench, which has no training split, every method including the answer-key arms learns by 5-fold cross-fitting with all paraphrases of a question kept in one fold, and the calibration fifth of each training fold is drawn by paraphrase group. The feedback rows of each task are split 80 / 20 into training and calibration rows with a fixed permutation. Stochastic methods run with seeds 0, 1 and 2. Gains are paired differences in per-row correctness between two methods on the same rows, in points. Their 95% intervals come from a bootstrap that resamples the training seeds of each side with replacement and then resamples rows by cluster: tasks on the public menu (which is macro-averaged over tasks) and DecisionBench, cases on typed-decisions, paraphrase groups on JevBench. With three seeds the seed component is itself an underestimate, so the intervals are still optimistic. Two-sided p-values come from the same bootstrap and are Holm-adjusted within each family

of comparisons a reader is likely to count (a method against the base model across benchmarks; Peekaboo against each answer-key arm across benchmarks; the KL contrasts). "Non-inferior" means the interval's lower bound is above -2 points. Same-seed reruns on different machines move individual results by up to 3 points (the earlier grid's typed-decisions seed 2 moved by 3.1), so the three-seed means, not single seeds, are what reproduce. Accuracy, ECE (15 equal-width bins, as in the README), NLL, Brier, AURC, macro-F1 and the per-benchmark slices come from the harness's `score_metrics`.

5.3 Methods compared

- **Released model**: Vega 0.8B, Vega 4B, Laya, replayed on cached inputs and checked against the official prediction call and the README numbers. On typed-decisions, Vega also with its shipped adapters off.
- **Their own recipe**: for Vega, VegaML's TTT head trained on the labelled feedback rows of each task, under the vegaml 0.8.0 rule (a head replaces the engine where its cross-validated accuracy beats chance) and the 0.12.3 rule (where it also beats the engine on the same rows); for Laya, the released `laya-typed-decisions` checkpoint on every benchmark, in-domain only on typed-decisions.
- **Answer-key add-on, same recipe**: the same adapter (Vega) or head copy (Laya) trained with cross-entropy on the labels of the same rows, the same KL term, eight passes, then label-based temperature scaling; also run for one pass. In the first version the Laya head was trained without the KL term; it now matches Vega's.
- **Answer-key add-on, early-stopped**: the same run with the epoch chosen by argmax accuracy on the calibration rows' labels. This is the tuned labelled reference the first version lacked.
- **Answer-key fine-tune, encoder unfrozen** (Laya): head and encoder trained with cross-entropy, the same KL term, rows and passes as the encoder-unfrozen Peekaboo; with and without early stopping.
- **Label prior**: one log-offset per (task, option text) on the released probabilities, no input features, trained with Peekaboo's own loop from verdicts (learning rate chosen on the calibration rows' verdicts) and, separately, from the labels.
- **Released model + temperature**: the released model with the honesty fix applied to its own sampled answers on its calibration rows, and with a label-fitted temperature.
- **Peekaboo** with eight passes and with one pass; head only and encoder unfrozen on Laya; without the KL term; with the switch. (The first version's pooled variant, one add-on trained on every benchmark's feedback, is not repeated here.)
- **Realistic feedback**: the released model's best answers rated once and learned off-policy (a verdict of 1 is a label, a verdict of 0 rules that answer out); its sampled answers rated once and learned with clipped importance weights; eight-pass Peekaboo with 20% of verdicts flipped, with only half of the answers rated, and with every wrong answer reported but only 20% of right ones.

5.4 Compute and reproducibility

The grid (314 steps: caches, replays and every method and seed) ran in about ten hours on one NVIDIA A40 from cached backbone features; the seed counts that budget allowed are listed in Section 7. One script, `experiments/peekaboo_remaining/run.py`, runs every step, skips finished ones and retries failures; `analysis.py` computes everything in this paper from the per-row records; `paper/make_figures.py` writes every table, figure and number in the text, so the prose cannot drift from the results. The first version's results stay in the repository under `output/`.

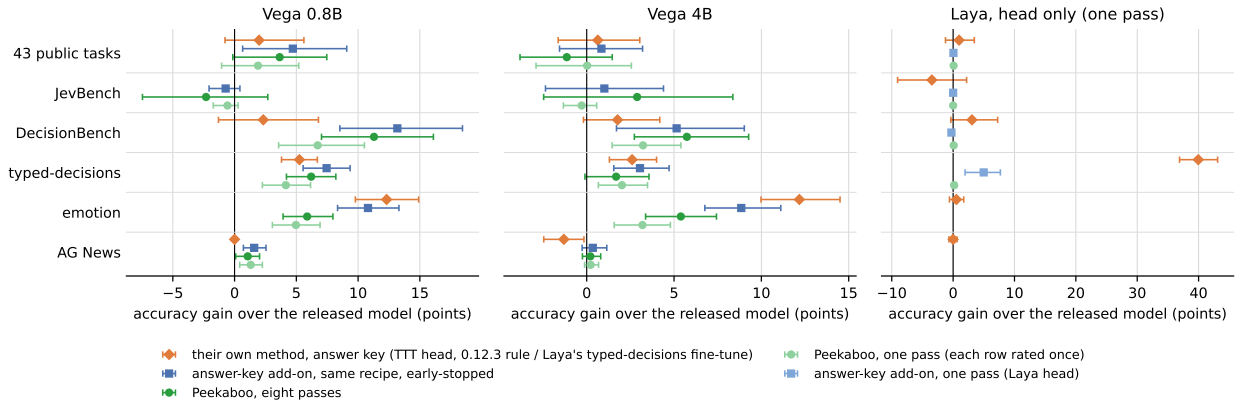


Figure 2: Gain over the released model on the six benchmarks, in accuracy points with 95% intervals (seeds and row clusters resampled). Each row is a benchmark; each marker a method. Orange diamonds: the model's own labelled-data recipe (VegaML's TTT head under the 0.12.3 rule; Laya's typed-decisions fine-tune, in-domain only on typed-decisions). Blue squares: our add-on trained on the labels, early-stopped. Dark green: Peekaboo with eight passes; light green: Peekaboo with one pass, each row rated once. On Laya the head-only arms were run for one pass only (light blue: the labelled head, one pass). On JevBench every method is cross-fitted.

Table 2: Accuracy, mean over 3 seeds (± s.d.) where a method is stochastic. JevBench has no training split: every method learns by 5-fold cross-fitting there, and pooled Peekaboo never sees it.

model	method	learns from	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
Vega 0.8B	released model	none	0.556	0.545	0.509	0.666	0.568	0.901
	TTT head, 0.8.0 rule	answer key	0.516	·	0.530	0.719	0.691	0.900
	TTT head, 0.12.3 rule	answer key	0.576	·	0.532	0.719	0.691	0.901
	answer-key adapter, same recipe	answer key	0.590 ±0.004	0.537 ±0.020	0.632 ±0.004	0.737 ±0.002	0.679 ±0.007	0.901 ±0.003
	answer-key adapter, early-stopped	answer key	0.603 ±0.010	0.538 ±0.005	0.641 ±0.010	0.740 ±0.003	0.676 ±0.006	0.917 ±0.002
	answer-key adapter, one pass	answer key	0.592 ±0.003	0.540 ±0.002	0.608 ±0.007	0.734 ±0.007	0.635 ±0.002	0.917 ±0.002
	Peekaboo, eight passes	verdicts	0.593 ±0.013	0.522 ±0.010	0.622 ±0.002	0.728 ±0.007	0.627 ±0.004	0.912 ±0.004
	Peekaboo, one pass	verdicts	0.575 ±0.008	0.540 ±0.002	0.576 ±0.011	0.708 ±0.003	0.618 ±0.006	0.914 ±0.002
	logged best answers, rated once	verdicts	·	·	0.583 ±0.009	0.726	0.603	·
	Peekaboo, no KL term	verdicts	0.582	0.517 ±0.014	0.609 ±0.004	0.709	0.622	0.916
Vega 4B	released model	none	0.718	0.667	0.636	0.729	0.575	0.928
	TTT head, 0.8.0 rule	answer key	0.678	·	0.640	0.757	0.697	0.915
	TTT head, 0.12.3 rule	answer key	0.724	·	0.654	0.755	0.697	0.915
	answer-key adapter, same recipe	answer key	0.706 ±0.002	0.706 ±0.011	0.687 ±0.007	0.769 ±0.001	0.640 ±0.001	0.924 ±0.003
	answer-key adapter, early-stopped	answer key	0.726 ±0.005	0.677 ±0.010	0.688 ±0.005	0.760 ±0.008	0.663 ±0.004	0.931 ±0.004
	answer-key adapter, one pass	answer key	0.723 ±0.003	0.667 ±0.000	0.683 ±0.009	0.775 ±0.008	0.631 ±0.004	0.931 ±0.003
	Peekaboo, eight passes	verdicts	0.706 ±0.005	0.696 ±0.005	0.694 ±0.005	0.746 ±0.010	0.629 ±0.009	0.930 ±0.002
	Peekaboo, one pass	verdicts	0.718 ±0.017	0.664 ±0.003	0.669 ±0.004	0.749 ±0.004	0.607 ±0.003	0.930 ±0.002
	logged best answers, rated once	verdicts	·	·	0.655 ±0.004	0.769	0.611	·
	Peekaboo, no KL term	verdicts	0.713	0.693	0.672 ±0.002	0.742	0.519	0.928
Laya	released model	none	0.540	0.576	0.406	0.362	0.599	0.930
	laya-typed-decisions (fine-tune on typed-decisions)	answer key	0.550	0.541	0.436	0.761	0.604	0.930
	answer-key head, one pass	answer key	0.541	0.576	0.403	0.412 ±0.011	·	·
	Peekaboo, head only, one pass	verdicts	0.541	0.576	0.406	0.364 ±0.003	·	·
	answer-key fine-tune, encoder unfrozen	answer key	0.587 ±0.009	0.535 ±0.025	0.560 ±0.016	0.750	0.899	·
	answer-key fine-tune, encoder unfrozen, early-stopped	answer key	0.584 ±0.003	0.595 ±0.005	0.554 ±0.005	0.750	0.905	·
	Peekaboo, encoder unfrozen, eight passes	verdicts	0.566 ±0.001	0.558 ±0.030	0.545 ±0.014	0.685	0.820	·
	Peekaboo, encoder unfrozen, one pass	verdicts	0.562 ±0.005	0.573 ±0.005	0.482 ±0.026	0.588	0.698	·

Table 3: Gain over the released model in accuracy points, with 95% intervals that resample seeds and row clusters; † marks differences that survive Holm adjustment within the method's family. JevBench has no training split: every method learns by 5-fold cross-fitting there, and pooled Peekaboo never sees it.

model	method	learns from	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
Vega 0.8B	released model	none	–	–	–	–	–	–
	TTT head, 0.8.0 rule	answer key	-4.1	·	+2.1	+5.3	+12.3	-0.2
	TTT head, 0.12.3 rule	answer key	+2.0	·	+2.3	+5.2	+12.3	+0.0
	answer-key adapter, same recipe	answer key	+3.4	-0.9	+12.3	+7.0	+11.1	+0.0
	answer-key adapter, early-stopped	answer key	+4.7	-0.7	+13.2	+7.4	+10.8	+1.6
	answer-key adapter, one pass	answer key	+3.6	-0.6	+10.0	+6.8	+6.6	+1.6
	Peekaboo, eight passes	verdicts	+3.7	-2.3	+11.3†	+6.2†	+5.9†	+1.1
	Peekaboo, one pass	verdicts	+1.9	-0.6	+6.7†	+4.1†	+5.0†	+1.3†
	logged best answers, rated once	verdicts	·	·	+7.4	+6.0	+3.5	·
	Peekaboo, no KL term	verdicts	+2.5	-2.9	+10.0	+4.2	+5.4	+1.5
Vega 4B	released model	none	–	–	–	–	–	–
	TTT head, 0.8.0 rule	answer key	-3.9	·	+0.4	+2.8	+12.2	-1.3
	TTT head, 0.12.3 rule	answer key	+0.6	·	+1.8	+2.6	+12.2	-1.3
	answer-key adapter, same recipe	answer key	-1.2	+3.9	+5.1	+4.0	+6.5	-0.3
	answer-key adapter, early-stopped	answer key	+0.8	+1.0	+5.1	+3.0	+8.8	+0.3
	answer-key adapter, one pass	answer key	+0.5	-0.0	+4.7	+4.6	+5.7	+0.3
	Peekaboo, eight passes	verdicts	-1.1	+2.9	+5.7†	+1.7	+5.4†	+0.2
	Peekaboo, one pass	verdicts	+0.0	-0.3	+3.2†	+2.0†	+3.2†	+0.2
	logged best answers, rated once	verdicts	·	·	+1.8	+4.0	+3.6	·
	Peekaboo, no KL term	verdicts	-0.5	+2.6	+3.5	+1.3	-5.6	+0.0
Laya	released model	none	–	–	–	–	–	–
	laya-typed-decisions (fine-tune on typed-decisions)	answer key	+1.0	-3.5	+3.1	+40.0	+0.5	+0.0
	answer-key head, one pass	answer key	+0.1	+0.0	-0.3	+5.0	·	·
	Peekaboo, head only, one pass	verdicts	+0.1	+0.0	+0.1	+0.2	·	·
	answer-key fine-tune, encoder unfrozen	answer key	+4.6	-4.0	+15.4	+38.8	+30.0	·
	answer-key fine-tune, encoder unfrozen, early-stopped	answer key	+4.3	+1.9	+14.8	+38.8	+30.6	·
	Peekaboo, encoder unfrozen, eight passes	verdicts	+2.6	-1.7	+14.0†	+32.3†	+22.2†	·
	Peekaboo, encoder unfrozen, one pass	verdicts	+2.1	-0.3	+7.7	+22.6	+9.9	·

Table 4: Head to head, in accuracy points with 95% intervals (seeds and row clusters resampled). Positive means the first method is better. †: survives Holm adjustment within the row's family; NI: non-inferior at a 2-point margin (lower bound above −2). TTT is scored under the vegaml 0.12.3 rule; for Laya "their method" is the typed-decisions fine-tune, in-domain only on typed-decisions.

model	comparison	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
Vega 0.8B							
	Peekaboo (8 passes) vs. their method	+1.7 [-1.6, +5.0]	–	+9.0 [+5.4, +12.6]	+0.9 [-0.7, +2.7]	-6.4 [-8.4, -4.3]	+1.1 [+0.1, +2.0]
	Peekaboo (8) vs. answer key, same recipe (8)	+0.3 [-2.8, +3.6]	-1.4 [-6.1, +3.0]	-1.0 [-3.0, +1.0]	-0.9 [-2.2, +0.5]	-5.2 [-7.1, -3.3]†	+1.0 [-0.1, +2.1] NI
	Peekaboo (8) vs. answer key, early-stopped	-1.1 [-4.0, +2.0]	-1.6 [-7.0, +3.5]	-1.9 [-4.1, +0.3]	-1.3 [-2.6, +0.1]	-4.9 [-6.8, -3.0]	-0.5 [-1.3, +0.2] NI
	Peekaboo (1 pass) vs. answer key (1 pass)	-1.7 [-3.7, +0.5]	-0.0 [-1.0, +1.0] NI	-3.2 [-5.8, -0.6]	-2.7 [-4.1, -1.2]	-1.7 [-3.0, -0.4]	-0.3 [-0.9, +0.4] NI
	Peekaboo, 1 pass vs. 8 passes	-1.8 [-4.4, +0.8]	+1.7 [-3.2, +7.2]	-4.6 [-8.1, -1.4]	-2.1 [-3.6, -0.4]	-0.9 [-2.4, +0.5]	+0.2 [-0.5, +1.0]
	logged best answers vs. Peekaboo (1 pass)	–	–	+0.7 [-2.4, +3.7]	+1.9 [+0.3, +3.5] NI	-1.5 [-4.1, +0.7]	–
	KL term vs. none	+1.1 [-1.5, +3.9]	+0.6 [-2.4, +3.7]	+1.2 [-0.6, +3.0]	+1.9 [+0.3, +3.5]	+0.5 [-1.0, +1.8]	-0.5 [-1.3, +0.3]
	with the switch vs. without	+0.0 [-2.2, +2.3]	+2.3 [-2.9, +7.7]	+0.0 [-1.2, +1.1]	+0.0 [-1.2, +1.3]	+0.0 [-1.1, +1.1]	+0.0 [-0.7, +0.7]
Vega 4B							
	Peekaboo (8 passes) vs. their method	-1.8 [-4.3, +0.7]	–	+4.0 [+1.5, +6.5]	-0.9 [-2.6, +0.9]	-6.8 [-8.9, -4.7]	+1.5 [+0.4, +2.6]
	Peekaboo (8) vs. answer key, same recipe (8)	+0.0 [-2.0, +2.0]	-1.0 [-5.0, +2.8]	+0.6 [-1.6, +2.7] NI	-2.3 [-3.8, -0.8]†	-1.1 [-3.2, +1.0]	+0.6 [-0.1, +1.3] NI
	Peekaboo (8) vs. answer key, early-stopped	-2.0 [-4.0, -0.2]	+1.9 [-2.7, +6.6]	+0.6 [-1.2, +2.3] NI	-1.4 [-2.9, +0.4]	-3.5 [-5.3, -1.7]	-0.1 [-0.8, +0.5] NI
	Peekaboo (1 pass) vs. answer key (1 pass)	-0.5 [-3.0, +1.4]	-0.3 [-1.4, +0.7] NI	-1.4 [-3.8, +0.8]	-2.6 [-4.0, -1.2]	-2.5 [-3.7, -1.2]	-0.1 [-0.7, +0.5] NI
	Peekaboo, 1 pass vs. 8 passes	+1.2 [-1.3, +3.5]	-3.2 [-8.7, +2.4]	-2.5 [-5.8, +0.0]	+0.3 [-1.2, +1.9]	-2.2 [-3.8, -0.5]	+0.0 [-0.6, +0.5]
	logged best answers vs. Peekaboo (1 pass)	–	–	-1.4 [-4.4, +1.9]	+2.0 [+0.7, +3.4] NI	+0.4 [-1.9, +2.6] NI	–
	KL term vs. none	-0.6 [-2.8, +1.4]	+0.3 [-3.1, +3.8]	+2.2 [+0.4, +4.2]	+0.4 [-1.5, +2.3]	+11.0 [+8.5, +13.5]†	+0.2 [-0.2, +0.8]
	with the switch vs. without	+0.0 [-1.3, +1.2]	-2.7 [-8.1, +2.5]	+0.0 [-1.1, +1.2]	-0.7 [-3.0, +1.5]	+0.0 [-1.5, +1.6]	+0.0 [-0.5, +0.4]
Laya							
	Peekaboo, head (1) vs. answer-key head (1)	+0.0 [-0.3, +0.3] NI	+0.0 [+0.0, +0.0] NI	+0.4 [+0.1, +0.7] NI	-4.8 [-7.5, -2.0]	–	–
	Peekaboo, encoder (8) vs. their fine-tune	+1.6 [-2.1, +5.3]	+1.7 [-4.7, +8.8]	+10.9 [+4.7, +16.7]†	-7.6 [-9.8, -5.4]†	+21.6 [+19.4, +24.0]†	–
	Peekaboo, encoder (8) vs. answer-key encoder, same recipe	-2.1 [-5.1, +0.7]	+2.3 [-3.9, +7.9]	-1.4 [-5.1, +2.0]	-6.5 [-8.7, -4.1]†	-7.9 [-9.6, -6.1]†	–
	Peekaboo, encoder (8) vs. answer-key encoder, early-stopped	-1.8 [-4.7, +0.9]	-3.6 [-8.7, +1.4]	-0.9 [-4.5, +2.0]	-6.5 [-8.7, -4.3]	-8.4 [-10.2, -6.7]	–
	Peekaboo, encoder: 1 pass vs. 8	-0.4 [-3.0, +2.1]	+1.4 [-4.4, +6.9]	-6.3 [-11.1, -1.9]	-9.7 [-12.1, -7.1]	-12.2 [-14.2, -10.3]	–

6 Results

6.1 Eight passes: verdicts against labels, and against the vendors' own recipes

Tables 2 and 3 give accuracy and gain over the released model for every arm; Table 4 the paired head-to-heads; Figure 2 the picture. Under the eight-pass protocol the verdict-trained add-on lands close to the same add-on trained on labels, and mostly below it. On DecisionBench, Peekaboo gains +11.3 points at 0.8B and +5.7 at 4B; the answer-key adapter with the same recipe gains +12.3 and +5.1, and early-stopped on its calibration labels +13.2 and +5.1. The paired difference against the early-stopped adapter is −1.9 [−4.1, +0.3] at 0.8B and +0.6 [−1.2, +2.3] at 4B. On typed-decisions it is −1.3 and −1.4, on the public menu −1.1 and −2.0, on AG News −0.5 and −0.1. Emotion is where labels pull furthest ahead at this capacity: −4.9 [−6.8, −3.0] and −3.5 [−5.3, −1.7]. Against the same-recipe adapter without early stopping every interval includes zero except emotion at 0.8B (−5.2) and typed-decisions at 4B (−2.3) (Table 4), which is what the $\pi(y)$ weighting predicts for a protocol that reveals most labels: the two arms are then optimising nearly the same objective. The first version read these gaps as "the bit is nearly as good as the label". The accurate reading is that eight rounds of verdicts on the same rows are nearly as good as the labels, because they mostly are the labels (Section 6.3); the one-pass numbers below are the ones that measure a bit.

Against VegaML's TTT head. Under the 0.12.3 rule, which keeps a head only where it beats the engine on its own fit rows, the TTT head gains +2.0 and +0.6 on the public menu (under the 0.8.0 rule it lost −4.1 and −3.9 points, the losses our first version counted as wins). Peekaboo minus the guarded TTT head is +1.7 [−1.6, +5.0] at 0.8B and −1.8 [−4.3, +0.7] at 4B; on DecisionBench +9.0 [+5.4, +12.6] and +4.0 [+1.5, +6.5]; on typed-decisions +0.9 and −0.9; on AG News +1.1 [+0.1, +2.0] and +1.5 [+0.4, +2.6]; on emotion −6.4 and −6.8. So the verdict-trained adapter is ahead of VegaML's labelled recipe on DecisionBench by a wide margin and on AG News by about a point, level on the public menu and typed-decisions, and well behind on emotion. On DecisionBench part of the margin is coverage: the TTT head answers only 394 of 1,075 rows (Table 8). At 0.8B almost all of the margin sits on the rows it leaves to the engine (+12.8 there against +2.8 [−1.8, +7.0] on covered rows); at 4B Peekaboo is ahead on both (+6.0 [+0.8, +11.4] covered, +5.0 uncovered). The labelled adapter has the same advantage over TTT there (+3.8, +5.6 on covered rows), so this is a property of an in-engine adapter that reads the option text, not of the feedback signal. Emotion goes to TTT by a wide margin at both sizes: a six-way single-label task with 3,200 labelled rows is where a logistic head on good features is at its best.

JevBench. With every arm cross-fitted, the answer-key adapter is defined on JevBench too, and it changes the first version's claim that Peekaboo was the only method to improve Vega 4B there. At 4B Peekaboo gains +2.9 [−2.5, +8.4] and the labelled adapter +3.9 [−2.3, +10.0]; at 0.8B both lose (−2.3 and −0.9), and the switch returns Peekaboo to the released model (+0.0). JevBench has 231 rows and 38-to-42-row calibration folds; none of its intervals is narrow.

6.2 One rating per row, and feedback as a deployment delivers it

Figure 3 shows what a single pass keeps. On DecisionBench one-pass Peekaboo gains +6.7 [+3.6, +10.5] at 0.8B and +3.2 [+1.5, +5.4] at 4B, against +11.3 and +5.7 with eight passes; the one-pass labelled adapter gains +10.0 and +4.7. At matched epochs the gap between verdicts and labels is −3.2 [−5.8, −0.6] and −1.4 [−3.8, +0.8], wider than with eight passes. On typed-decisions one pass gives +4.1 and +2.0 (labels: +6.8, +4.6; verdicts significantly behind at both sizes); on the public menu +1.9 and +0.0; on emotion +5.0 and +3.2; on AG News +1.3 and +0.2. One pass keeps about half to two thirds of the eight-pass gain on DecisionBench and the public menu and more on emotion.

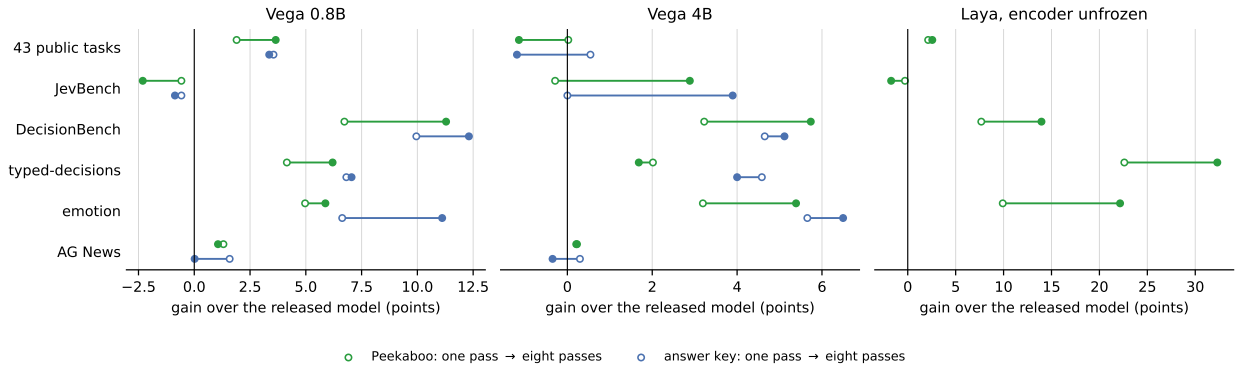


Figure 3: One pass (open markers) against eight (filled), for Peekaboo (green) and the answer-key add-on (blue), as gain over the released model. One pass rates each row once and takes one eighth of the gradient steps. Laya: the encoder-unfrozen Peekaboo only.

Table 5: Feedback as a deployment delivers it: gain over the released model in points (95% intervals), on the three benchmarks with the realistic-feedback arms.

feedback	DecisionBench			typed-decisions			emotion		
	Vega 0.8B	Vega 4B	Laya (head)	Vega 0.8B	Vega 4B	Laya (head)	Vega 0.8B	Vega 4B	Laya (head)
eight passes, clean verdicts	+11.3 [+7.0, +16.1]	+5.7 [+2.7, +9.3]	–	+6.2 [+4.2, +8.2]	+1.7 [-0.1, +3.6]	–	+5.9 [+3.9, +8.0]	+5.4 [+3.4, +7.4]	–
one pass, clean verdicts	+6.7 [+3.6, +10.5]	+3.2 [+1.5, +5.4]	+0.1 [+0.0, +0.3]	+4.1 [+2.2, +6.1]	+2.0 [+0.7, +3.5]	+0.2 [-0.2, +0.5]	+5.0 [+3.1, +6.9]	+3.2 [+1.6, +4.8]	–
logged: the released model's best answers, rated once	+7.4 [+3.4, +12.2]	+1.8 [-1.4, +5.3]	–	+6.0 [+3.9, +8.2]	+4.0 [+2.5, +5.7]	–	+3.5 [+0.7, +6.2]	+3.6 [+1.2, +5.9]	–
logged: the released model's sampled answers, rated once (clipped IPS)	+8.3 [+4.4, +13.0]	+4.3 [+1.6, +7.3]	–	+5.6 [+3.5, +7.6]	+1.8 [+0.1, +3.8]	–	+6.6 [+4.6, +8.6]	+4.5 [+2.5, +6.6]	–
20% of verdicts flipped (eight passes)	+9.1 [+5.8, +12.8]	+4.2 [+1.7, +7.3]	–	+5.9 [+4.2, +7.5]	+2.8 [+1.2, +4.3]	–	+4.7 [+3.1, +6.3]	+4.7 [+2.7, +6.8]	–
half of the answers rated (eight passes)	+10.3 [+6.2, +14.8]	+4.7 [+1.6, +8.3]	–	+5.2 [+3.1, +7.5]	+3.5 [+1.8, +5.3]	–	+6.5 [+4.2, +8.9]	+0.0 [+0.0, +0.0]	–
every wrong answer reported, 20% of right ones (eight passes)	+10.0 [+6.0, +14.7]	+5.7 [+2.7, +9.1]	–	+7.0 [+5.0, +9.0]	+4.2 [+2.4, +5.9]	–	+6.7 [+4.5, +9.0]	+2.7 [+0.9, +4.5]	–
answer key, one pass	+10.0 [+6.3, +13.9]	+4.7 [+2.1, +7.4]	-0.3 [-0.7, +0.0]	+6.8 [+4.8, +8.8]	+4.6 [+3.0, +6.4]	+5.0 [+1.9, +7.7]	+6.6 [+4.5, +8.8]	+5.7 [+3.8, +7.7]	–
answer key, eight passes	+12.3 [+7.8, +17.5]	+5.1 [+1.4, +9.1]	–	+7.0 [+5.0, +9.1]	+4.0 [+2.5, +5.6]	–	+11.1 [+8.6, +13.6]	+6.5 [+4.3, +8.8]	–

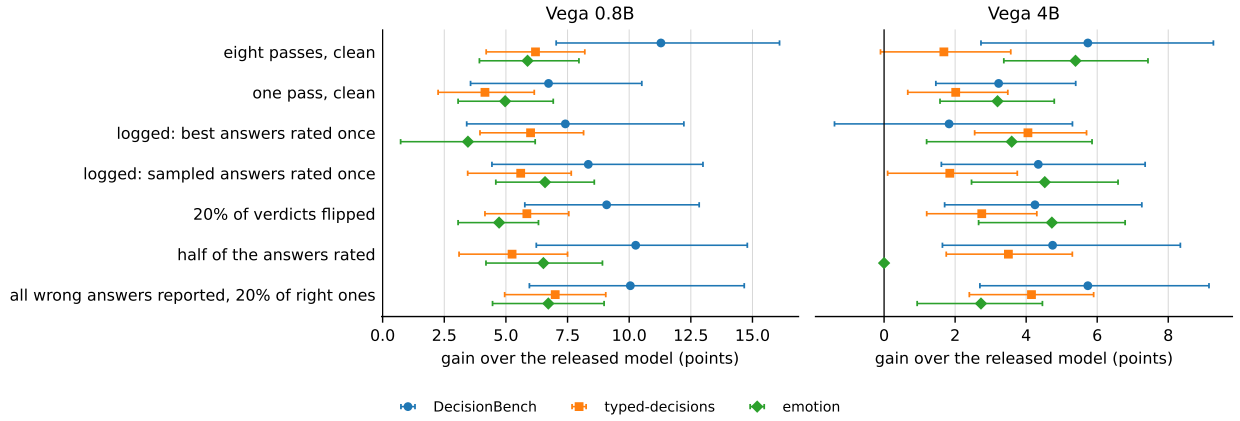


Figure 4: Feedback as a deployment delivers it, on Vega: gain over the released model with 95% intervals for the clean eight-pass and one-pass arms, the two logged arms (one verdict per row on the released model's best or sampled answers, trained off-policy), and eight-pass Peekaboo with flipped, missing or complaint-skewed verdicts. Three seeds on DecisionBench, one on typed-decisions and emotion.

One pass confounds two things: each row is rated once, and the model takes one eighth of the gradient steps. The logged arms separate them. Each row of the pool is answered once by the released model, rated once, and the add-on is then trained on that fixed log for eight passes, off-policy. With the released model's sampled answers and clipped importance weights, this recovers most of the eight-pass on-policy gain: +8.3 $[+4.4, +13.0]$ on DecisionBench at 0.8B and +4.3 at 4B, against +11.3 and +5.7; on typed-decisions and emotion it is level with eight on-policy passes (-0.6, +0.7 at 0.8B; +0.2, -0.9 at 4B), and above one on-

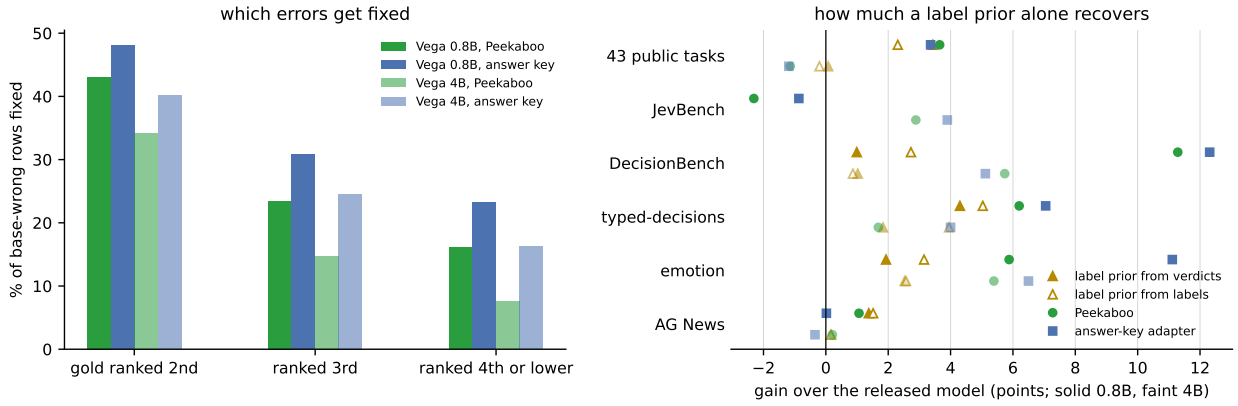


Figure 5: Left: share of the released model's wrong rows that eight-pass Peekaboo and the answer-key adapter fix, by the released model's rank of the gold option (mean over benchmarks). Right: gain over the released model of a per-task label prior with no input features, trained from verdicts (filled triangles) or labels (open), beside Peekaboo and the answer-key adapter.

policy pass everywhere it was run. So most of the difference between one pass and eight is optimisation, not information: a single verdict per row, trained to convergence, carries most of what eight on-policy verdicts do on these models. It is still not the label. Against the early-stopped labelled adapter the logged arm is −4.8 [−7.7, −2.0] on DecisionBench at 0.8B, −1.8 on typed-decisions and −4.2 on emotion; at 4B −0.8, −1.2 and −4.3.

Table 5 and Figure 4 collect the deployment question. A system that serves its best answer, rather than a sample, logs only that answer's verdict: a 1 is a label, a 0 rules one option out. Learning from that log gains +7.4 [+3.4, +12.2] on DecisionBench at 0.8B and +1.8 [−1.4, +5.3] at 4B, +6.0 and +4.0 on typed-decisions, +3.5 and +3.6 on emotion. Logging sampled answers instead, which costs some accuracy while the log is collected, is worth +0.9 and +2.5 points more on DecisionBench and +3.1 on emotion at 0.8B, and less on typed-decisions at 4B (−2.2). The eight-pass arm is robust to the kinds of damage real feedback has: with 20% of verdicts flipped it gains +9.1 and +4.2 on DecisionBench, with half of the answers unrated +10.3 and +4.7, and when every wrong answer is reported but only one in five right ones +10.0 and +5.7; the same holds on typed-decisions and emotion, with one exception, emotion at 4B with half of the answers unrated (+0.0, one seed; Table 5). Rewards are still computed from the gold answer in all of these; a human who is wrong about what was right is not simulated, beyond the flipped verdicts.

6.3 What the gain is made of

How much the protocol reveals. Table 6 (right) counts what eight on-policy passes tell the learner. On DecisionBench at 0.8B each training row produces 4.88 reward-1 events on average, each of which names the gold answer; from the released model's own probabilities, 8 draws would hit the gold at least once on a share 0.76 of the rows, and 0.21 of the rows have two options, where any verdict is the label. On typed-decisions the figures are 6.12 reveals per row and 0.94; on emotion 4.90 and 0.85. The first pass's reward, which is the only out-of-sample one, is 0.48 on DecisionBench against the released model's best-answer accuracy of 0.50 on the same rows: the exploration a live system would pay for is a few points of accuracy during the first pass, and nothing afterwards if it then serves the argmax.

A label prior with no input. Per-task log-offsets on the released probabilities, one per (task, option text), trained with Peekaboo's own loop from verdicts, gain +4.3 on typed-decisions at 0.8B and +1.8 at 4B, against

Table 6: What the gain is made of. Left: gain over the released model of a per-task label prior with no input features (log-offsets on the released probabilities), trained from verdicts with Peekaboo's loop and from labels; and Peekaboo minus that prior. Right: how often eight on-policy passes reveal the gold answer: reward-1 events per training row (each one names the gold answer), the share of rows the base policy would reveal in 8 draws, the share of two-option rows, and the first pass's reward against the released model's best-answer accuracy on the same rows.

model	benchmark	prior (verdicts)	prior (labels)	Peekaboo − prior	params	reveals / row	P(revealed, 8 draws)	2-option rows	pass-1 reward	base best-answer acc.
Vega 0.8B	43 public tasks	+3.4	+2.3	+0.2 [-3.0, +3.1]	536	5.1	0.88	30%	0.50	0.55
	JevBench	–	–	–	–	4.7	–	–	0.49	–
	DecisionBench	+1.0	+2.7	+10.3 [+6.3, +14.7]	28584	4.9	0.76	21%	0.48	0.50
	typed-decisions	+4.3	+5.0	+1.9 [+0.1, +3.6]	71	6.1	0.94	30%	0.72	0.72
	emotion	+1.9	+3.1	+3.9 [+2.1, +5.7]	6	4.9	0.85	0%	0.57	0.56
	AG News	+1.4	+1.5	-0.3 [-1.2, +0.5]	4	7.5	0.97	0%	0.92	0.92
Vega 4B	43 public tasks	+0.1	-0.2	-1.2 [-3.6, +0.9]	536	6.1	0.92	30%	0.68	0.70
	JevBench	–	–	–	–	5.9	–	–	0.64	–
	DecisionBench	+1.0	+0.9	+4.7 [+2.0, +7.8]	28584	5.6	0.82	21%	0.62	0.61
	typed-decisions	+1.8	+3.9	-0.2 [-1.8, +1.6]	71	6.4	0.96	30%	0.75	0.79
	emotion	+2.5	+2.6	+2.9 [+1.1, +4.7]	6	4.8	0.84	0%	0.57	0.57
	AG News	+0.2	+0.2	+0.0 [-0.4, +0.6]	4	7.4	0.96	0%	0.92	0.93
Laya	typed-decisions	+15.3	+17.4	–	71	–	–	–	–	–
	emotion	-1.4	-3.3	–	6	–	–	–	–	–
	AG News	-0.6	-0.1	–	4	–	–	–	–	–

Table 7: Which rows change. Share of the released model's wrong rows that a method fixes, by the released model's rank of the gold option, and share of its right rows the method breaks (mean over benchmarks with a training split and over seeds).

model	method	fixed, all	gold 2nd	gold 3rd	gold 4th+	broken
Vega 0.8B	Peekaboo (8 passes)	32%	43%	23%	13%	11%
Vega 0.8B	Peekaboo (1 pass)	26%	37%	13%	9%	8%
Vega 0.8B	answer key (8 passes)	40%	48%	31%	23%	13%
Vega 4B	Peekaboo (8 passes)	25%	34%	15%	8%	8%
Vega 4B	Peekaboo (1 pass)	18%	27%	9%	4%	6%
Vega 4B	answer key (8 passes)	32%	40%	25%	16%	10%
Laya	Peekaboo (1 pass)	0%	1%	0%	0%	0%
Laya	Peekaboo, encoder (8)	47%	58%	45%	32%	16%
Laya	answer-key encoder (8)	55%	65%	55%	44%	15%

Table 8: VegaML's TTT head only answers rows whose option set matches a fitted head ("covered"); elsewhere its output is the released engine's. Peekaboo minus TTT (0.8.0 rule) on covered and uncovered rows, and the answer-key adapter minus TTT on covered rows.

model	benchmark	rows covered	Peekaboo − TTT, covered	uncovered	answer key − TTT, covered
Vega 0.8B	43 public tasks	1945 / 2535	+7.9 [+2.5, +14.4]	+7.0 [-0.6, +16.8]	+6.8 [+1.2, +13.3]
Vega 0.8B	DecisionBench	394 / 1075	+2.8 [-1.8, +7.0]	+12.8 [+8.5, +17.6]	+3.8 [-1.1, +8.3]
Vega 0.8B	typed-decisions	2000 / 2000	+0.9 [-1.2, +3.1]	+nan [+nan, +nan]	+1.7 [-0.3, +3.9]
Vega 0.8B	emotion	1503 / 1503	-6.4 [-8.4, -4.3]	+nan [+nan, +nan]	-1.2 [-3.3, +0.9]
Vega 0.8B	AG News	1912 / 1912	+1.2 [-0.0, +2.5]	+nan [+nan, +nan]	+0.2 [-1.1, +1.4]
Vega 4B	43 public tasks	2125 / 2535	+3.1 [-1.0, +7.4]	+1.1 [+nan, +nan]	+2.8 [-1.1, +7.1]
Vega 4B	DecisionBench	394 / 1075	+6.0 [+0.8, +11.4]	+5.0 [+2.2, +8.1]	+5.6 [+0.6, +10.8]
Vega 4B	typed-decisions	2000 / 2000	-1.1 [-3.3, +1.1]	+nan [+nan, +nan]	+1.2 [-0.7, +3.0]
Vega 4B	emotion	1503 / 1503	-6.8 [-8.9, -4.7]	+nan [+nan, +nan]	-5.7 [-7.7, -3.7]
Vega 4B	AG News	1912 / 1912	+1.5 [+0.4, +2.6]	+nan [+nan, +nan]	+1.0 [-0.2, +2.1]

Table 9: Benchmark artifacts. Gain over the released model with every task, and without the tasks whose gold answer an adapter can learn without reading the input (public: truthfulqa_mc1, pubmedqa; DecisionBench: moderation-civil-comments, finqa-score).

model	method	public, all 43	public, 41 tasks	DecisionBench, all 43	DecisionBench, 40 tasks
Vega 0.8B	Peekaboo (8)	+3.7 [-0.1, +7.5]	+1.9 [-1.4, +5.1]	+11.3 [+7.0, +16.1]	+9.3 [+5.5, +14.0]
Vega 0.8B	Peekaboo (1)	+1.9 [-1.1, +5.2]	+0.5 [-2.1, +3.1]	+6.7 [+3.6, +10.5]	+5.9 [+2.8, +9.7]
Vega 0.8B	answer key	+3.4 [-0.9, +8.1]	+1.5 [-2.3, +5.3]	+12.3 [+7.8, +17.5]	+10.2 [+6.0, +15.1]
Vega 0.8B	TTT (0.12.3)	+2.0 [-0.8, +5.6]	+1.0 [-1.4, +3.5]	+2.3 [-1.3, +6.8]	+1.3 [-1.9, +5.5]
Vega 4B	Peekaboo (8)	-1.1 [-3.8, +1.5]	-1.8 [-4.5, +0.8]	+5.7 [+2.7, +9.3]	+4.6 [+2.1, +7.1]
Vega 4B	Peekaboo (1)	+0.0 [-2.9, +2.6]	-0.4 [-3.6, +2.1]	+3.2 [+1.5, +5.4]	+3.4 [+1.6, +5.7]
Vega 4B	answer key	-1.2 [-4.1, +1.8]	-2.2 [-5.0, +0.6]	+5.1 [+1.4, +9.1]	+3.7 [+0.5, +7.3]
Vega 4B	TTT (0.12.3)	+0.6 [-1.6, +3.0]	+0.1 [-2.1, +2.2]	+1.8 [-0.2, +4.2]	+1.3 [-0.6, +3.5]
Laya	Peekaboo (1)	+0.1 [-0.1, +0.3]	+0.1 [-0.1, +0.3]	+0.1 [+0.0, +0.3]	+0.1 [+0.0, +0.3]
Laya	Peekaboo, encoder	+2.6 [-0.8, +6.0]	+2.0 [-1.4, +5.5]	+14.0 [+8.8, +19.2]	+12.9 [+7.7, +18.1]
Laya	their fine-tune	+1.0 [-1.2, +3.5]	+0.8 [-1.6, +3.4]	+3.1 [-0.4, +7.3]	+2.7 [-1.1, +7.1]

Peekaboo's +6.2 and +1.7; Peekaboo minus the prior is +1.9 [+0.1, +3.6] and −0.2 [−1.8, +1.6]. About two thirds of the typed-decisions gain at 0.8B, and all of it at 4B, is a prior shift that 71 numbers can learn from verdicts. On the public menu the prior gains +3.4 and +0.1 (Peekaboo minus prior +0.2, −1.2). On DecisionBench the prior gains +1.0 and +1.0 against Peekaboo's +11.3 and +5.7, so there the adapter learns something the option text alone does not give (Peekaboo minus prior +10.3 [+6.3, +14.7], +4.7 [+2.0, +7.8]); on emotion the prior gains +1.9 and +2.5 against +5.9 and +5.4. DecisionBench and emotion are the benchmarks on which the paper's claim about learning from verdicts rests; typed-decisions and the public menu are largely prior correction, which verdicts also deliver, but which a 71-parameter model delivers too.

Which rows change. Table 7 and Figure 5 (left) test the prediction of the $\pi(y)$ weighting. Over the benchmarks with a training split, eight-pass Peekaboo fixes 32% of the released model's wrong rows at 0.8B and breaks 11% of its right ones; the labelled adapter fixes 40% and breaks 13%. Where the released model ranked the gold option second, Peekaboo fixes 43% of rows against the adapter's 48%; where it ranked it fourth or lower, 13% against 23%. The verdict-trained adapter is the more conservative learner: it fixes fewer of the deep errors, as a $\pi(y)$-weighted objective must, and breaks fewer correct answers. The near-parity of the headline accuracies is those two differences cancelling.

Artifacts. Table 9 repeats the public-menu and DecisionBench gains without the four tasks whose gold answer can be learned from the option list alone. On the public menu at 0.8B Peekaboo's gain goes from +3.7 to +1.9 [−1.4, +5.1]; at 4B from −1.1 to −1.8. On DecisionBench from +11.3 to +9.3 [+5.5, +14.0] and from +5.7 to +4.6 [+2.1, +7.1]. The labelled adapter and TTT profit from the same tasks, so the comparisons between methods move little; the gains over the released model are what the artifacts inflate. The per-task tables in Appendix C show where the rest of the DecisionBench gain sits.

6.4 Calibration

Before the honesty fix the trained add-on is badly over-confident: ECE 0.257 on DecisionBench at 0.8B, 0.237 at 4B, 0.293 and 0.283 on emotion (Table 10, Figure 7). One temperature fitted from verdicts brings these to 0.053, 0.047, 0.052 and 0.049. The control the first version lacked is the released model with the same temperature fitted on its own calibration rows: its ECE is 0.072 at 0.8B and 0.053 at 4B on DecisionBench, against the released model's 0.101 and 0.095. Much of Peekaboo's ECE advantage over the released model is therefore the temperature, not the adapter, and we no longer claim otherwise. On proper scores and on

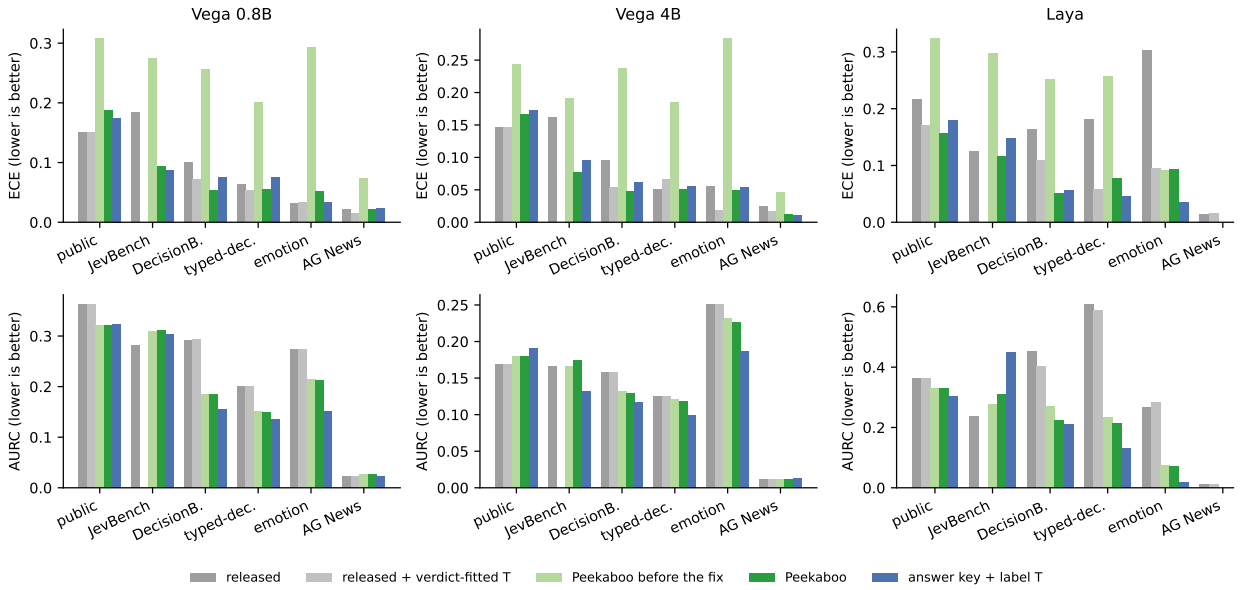


Figure 6: ECE (top) and area under the risk-coverage curve (bottom), lower is better, for the released model, the released model with a temperature fitted from verdicts on its own calibration rows, Peekaboo before and after the honesty fix, and the answer-key adapter with a label-fitted temperature. Laya: the encoder-unfrozen arms.

Table 10: Calibration and proper scores (lower is better; mean over seeds). "+ T" rows apply one temperature to the released model, fitted on its own calibration rows from verdicts on sampled answers (mean over 10 draws) or from labels.

model	method	public			JevBench			DecisionB.			typed-dec.			emotion			AG News		
		ECE	NLL	AURC	ECE	NLL	AURC	ECE	NLL	AURC	ECE	NLL	AURC	ECE	NLL	AURC	ECE	NLL	AURC
Vega 0.8B																			
	released model	0.150	1.100	0.363	0.184	1.262	0.281	0.101	1.631	0.291	0.063	0.753	0.200	0.031	1.193	0.274	0.021	0.291	0.023
	released + T from verdicts	0.151	1.090	0.363	–	–	–	0.072	1.509	0.294	0.053	0.747	0.200	0.033	1.192	0.274	0.016	0.289	0.023
	released + T from labels	0.148	1.088	0.363	–	–	–	0.089	1.515	0.296	0.050	0.745	0.201	0.036	1.191	0.274	0.016	0.289	0.023
	Peekaboo before the honesty fix	0.308	2.735	0.321	0.275	1.758	0.308	0.257	3.547	0.186	0.200	1.354	0.151	0.293	3.593	0.215	0.073	0.707	0.026
	Peekaboo	0.187	1.114	0.322	0.093	1.036	0.312	0.053	1.316	0.185	0.055	0.690	0.150	0.052	1.057	0.213	0.023	0.281	0.027
	Peekaboo, one pass	0.177	1.102	0.342	0.079	1.006	0.287	0.059	1.376	0.212	0.059	0.714	0.160	0.046	1.045	0.211	0.012	0.256	0.020
	answer key + label temperature	0.174	1.073	0.323	0.086	1.053	0.304	0.075	1.178	0.155	0.076	0.685	0.135	0.033	0.911	0.151	0.024	0.291	0.023
Vega 4B																			
	released model	0.146	0.768	0.169	0.162	0.959	0.166	0.095	1.423	0.157	0.051	0.631	0.126	0.056	1.167	0.251	0.025	0.224	0.012
	released + T from verdicts	0.147	0.761	0.169	–	–	–	0.053	1.206	0.158	0.066	0.643	0.126	0.019	1.148	0.250	0.017	0.214	0.012
	released + T from labels	0.153	0.764	0.169	–	–	–	0.067	1.203	0.159	0.053	0.632	0.126	0.019	1.148	0.250	0.012	0.211	0.012
	Peekaboo before the honesty fix	0.244	2.374	0.180	0.192	1.316	0.166	0.237	3.328	0.132	0.185	1.237	0.121	0.283	2.981	0.232	0.046	0.407	0.012
	Peekaboo	0.167	0.836	0.179	0.076	0.783	0.174	0.047	1.095	0.129	0.051	0.637	0.118	0.049	1.087	0.226	0.012	0.208	0.012
	Peekaboo, one pass	0.143	0.775	0.171	0.064	0.776	0.178	0.040	1.135	0.144	0.055	0.643	0.123	0.028	1.115	0.235	0.013	0.208	0.013
	answer key + label temperature	0.173	0.840	0.191	0.096	0.679	0.131	0.062	1.051	0.116	0.056	0.567	0.100	0.054	0.996	0.187	0.011	0.220	0.013
Laya																			
	released model	0.217	1.651	0.365	0.125	0.928	0.236	0.165	2.062	0.454	0.182	1.361	0.610	0.303	2.082	0.267	0.015	0.206	0.012
	released + T from verdicts	0.171	1.189	0.365	–	–	–	0.110	1.631	0.404	0.058	1.207	0.590	0.095	1.176	0.282	0.016	0.206	0.012
	released + T from labels	0.174	1.159	0.361	–	–	–	0.103	1.631	0.404	0.059	1.205	0.588	0.097	1.177	0.282	0.017	0.206	0.012
	Peekaboo, one pass	0.163	1.111	0.364	0.107	0.911	0.245	0.075	1.575	0.389	0.034	1.194	0.563	–	–	–	–	–	–
	Peekaboo, encoder unfrozen	0.157	1.088	0.329	0.115	1.024	0.309	0.052	1.392	0.223	0.077	0.827	0.213	0.094	0.665	0.070	–	–	–
	answer-key encoder + label temperature	0.179	1.025	0.303	0.148	1.085	0.448	0.057	1.296	0.211	0.045	0.636	0.129	0.035	0.313	0.018	–	–	–

ranking the picture is less flattering to the verdict-trained arm: on DecisionBench at 0.8B its NLL is 1.316 against the labelled adapter's 1.178, its AURC 0.185 against 0.155 (released model: 1.631, 0.291). The temperature leaves the ranking of rows by confidence unchanged, so AURC measures what the adapter itself did to the confidence ordering; against the released model it is better on DecisionBench, typed-decisions and emotion at both sizes, and worse on JevBench at both sizes and on the public menu at 4B (Figure 6). The fitted temperatures range from 1.3 to 10.3 with a median of 3.8 (Figure 8); part of each is inherited from the

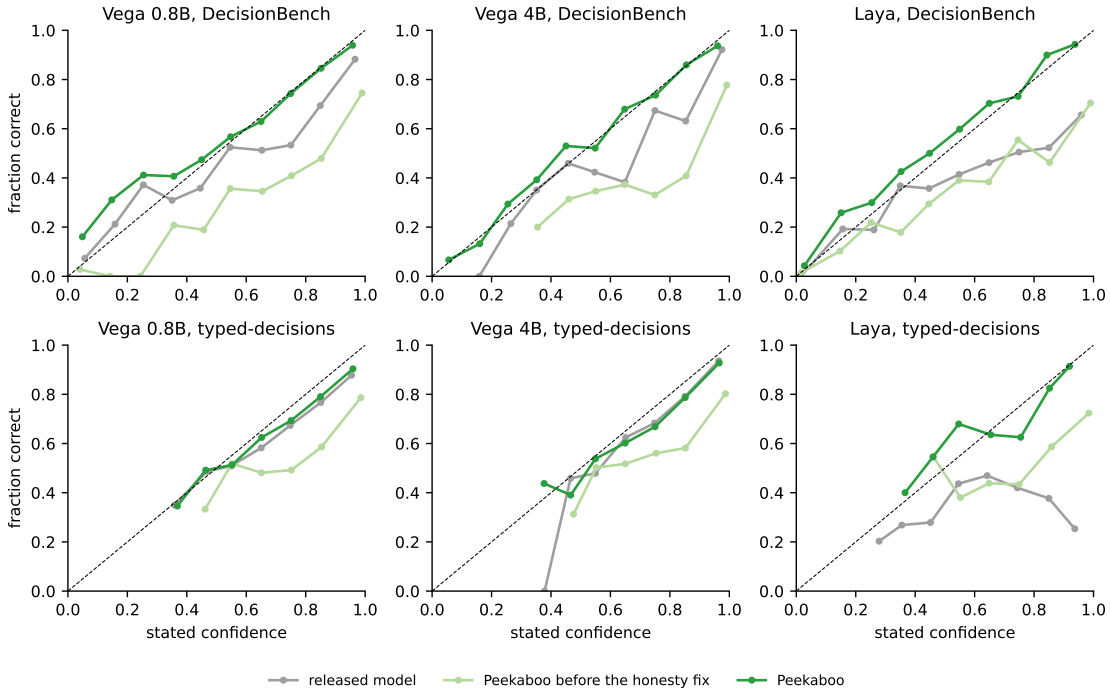


Figure 7: Reliability diagrams (10 bins on the confidence of the predicted option) on DecisionBench and typed-decisions: released model, Peekaboo before the honesty fix, Peekaboo after it (Laya: encoder unfrozen).

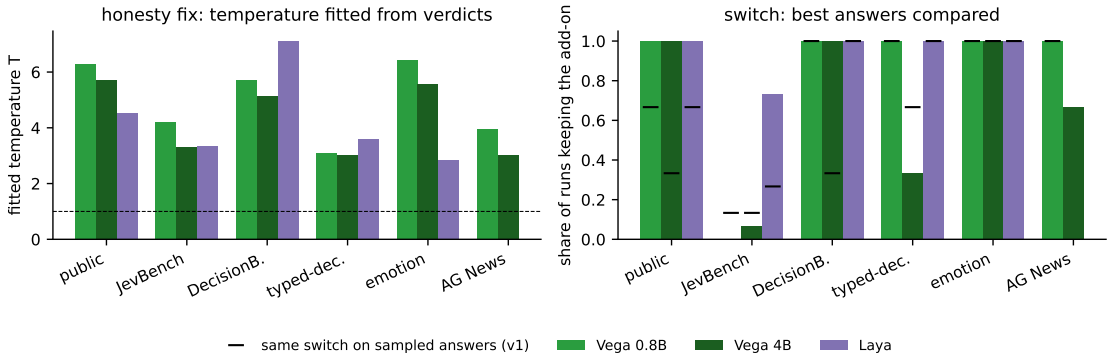


Figure 8: Left: the temperature fitted from verdicts per model and benchmark (dashed line at $T = 1$; Laya: encoder unfrozen). Right: the share of runs in which the switch, comparing best answers on the calibration rows, kept the add-on; black ticks show the first version's switch on the same runs, which compared sampled answers.

released model, which itself prefers a temperature above 1 on most benchmarks (2.0 on DecisionBench at 0.8B).

6.5 The KL term and the switch

Removing the KL term costs eight-pass Peekaboo +1.2 $[-0.6, +3.0]$ points on DecisionBench at 0.8B and +2.2 [+0.4, +4.2] at 4B (positive means the KL term helps; Table 4), +1.9 and +0.4 on typed-decisions, +1.1 and −0.6 on the public menu, and +0.5 and +11.0 on emotion. The term is worth a point or two in most cells and is decisive in one, emotion at 4B, where the run without it collapsed. Outside DecisionBench the no-KL

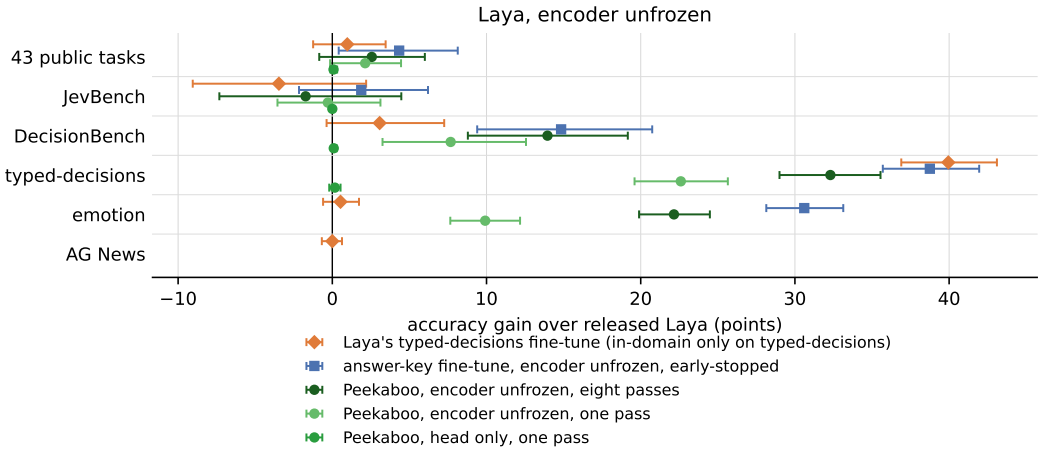


Figure 9: Laya with the encoder unfrozen: gain over released Laya of the typed-decisions fine-tune (in-domain only on typed-decisions), the matched answer-key fine-tune (same rows, passes and KL term, early-stopped), Peekaboo with eight passes and with one pass, and head-only Peekaboo with one pass. Encoder arms have three seeds on the public menu and JevBench, two on DecisionBench and one on typed-decisions and emotion; AG News was not run.

arm has one seed, so these cells say little on their own; the first version's "where the method lives" was too strong.

The switch now compares best answers. Where eight-pass Peekaboo hurts, on JevBench at 0.8B, it keeps the add-on in none of the folds, so the gain with the switch is +0.0 against −2.3 without. At 0.8B it keeps the add-on in every run on every other benchmark, and at 4B on DecisionBench, the public menu and emotion. It costs a real gain in one place, JevBench at 4B, where it keeps the add-on in 7% of folds and gives up 2.7 points, and drops a small, non-significant gain on typed-decisions at 4B (−0.7). The first version's switch, computed on the same runs, kept the add-on in 33% of the DecisionBench runs at 4B where the new one keeps 100%; the two statistics disagree in the direction Section 4.5 describes.

6.6 Laya: head, encoder, and the matched fine-tune

With the encoder frozen, a trainable copy of Laya's head learns a prior shift and little else; in the first version's grid, eight passes gave +13.2 points on typed-decisions where a 71-offset prior from the same verdicts gave +15.4, and in this grid one pass gives +0.2. With the encoder unfrozen, verdicts move the whole model: +32.3 [+29.0, +35.5] on typed-decisions with eight passes, +22.6 with one, +22.2 on emotion, +14.0 on DecisionBench and +2.6 on the public menu. The comparison the first version lacked is the same fine-tune on labels: cross-entropy, same rows, passes and KL term, encoder unfrozen. It gains +38.8 on typed-decisions, +30.0 on emotion, +15.4 on DecisionBench and +4.6 on the public menu; Peekaboo minus this arm is −6.5 [−8.7, −4.1] on typed-decisions, −7.9 [−9.6, −6.1] on emotion, −1.4 on DecisionBench and −2.1 on the public menu. At matched capacity and data, labels beat eight rounds of verdicts clearly on typed-decisions and emotion; on DecisionBench and the public menu the gap is one to two points and within the noise, and on JevBench the verdict-trained encoder is ahead (+2.3 [−3.9, +7.9]). With one pass the encoder arm falls well behind the labelled one: −16.2 on typed-decisions, −20.1 on emotion, −7.7 on DecisionBench. On Laya the logged arms were not run. On typed-decisions the matched fine-tune reaches 0.750 against the released `laya-typed-decisions` checkpoint's 0.761, so our labelled arm is a fair stand-in for Laya's own recipe. Away from typed-decisions the released fine-tune is a transfer model that never saw the benchmark, and in-domain training of any kind beats it: Peekaboo with the encoder is +21.6 above it on emotion and +10.9 on DecisionBench, the labelled encoder arm +29.5 and +12.3. Those numbers say that fine-tuning on

Table 11: typed-decisions on Vega with the shipped `typed_decisions` adapter, which was fitted with labels on this feedback pool, switched off. Accuracy (mean over seeds) and gain over the corresponding released model.

model	method	accuracy, adapters on	gain	accuracy, adapters off	gain
Vega 0.8B	released engine	0.666	–	0.377	–
Vega 0.8B	answer-key adapter (8)	0.737 ±0.002	+7.0 [+5.0, +9.1]	0.693 ±0.013	+31.6 [+28.9, +34.4]
Vega 0.8B	Peekaboo (8)	0.728 ±0.007	+6.2 [+4.2, +8.2]	0.507 ±0.028	+13.0 [+9.8, +16.5]
Vega 0.8B	Peekaboo (1)	0.708 ±0.003	+4.1 [+2.2, +6.1]	0.498 ±0.007	+12.1 [+10.1, +14.2]
Vega 4B	released engine	0.729	–	0.440	–
Vega 4B	answer-key adapter (8)	0.769 ±0.001	+4.0 [+2.5, +5.6]	0.748 ±0.006	+30.8 [+27.8, +33.9]
Vega 4B	Peekaboo (8)	0.746 ±0.010	+1.7 [-0.1, +3.6]	0.685 ±0.027	+24.6 [+21.0, +28.5]
Vega 4B	Peekaboo (1)	0.749 ±0.004	+2.0 [+0.7, +3.5]	0.661 ±0.004	+22.2 [+19.4, +25.0]

a benchmark's own rows helps on that benchmark; they say nothing about verdicts against labels, and we do not use them to.

6.7 typed-decisions without Vega's shipped adapter

Vega's released `typed_decisions` adapter was fitted on the pool our methods learn from, so on this benchmark "released Vega" already carries those labels. With the shipped adapters switched off the released engine scores 0.377 at 0.8B and 0.440 at 4B (Table 11). From that start eight-pass Peekaboo gains +13.0 [+9.8, +16.5] and +24.6 [+21.0, +28.5], the labelled adapter +31.6 and +30.8, and one-pass Peekaboo +12.1 and +22.2. The gap between verdicts and labels, −18.6 at 0.8B and −6.3 at 4B, is far wider than with the shipped adapter on, which is the clearest demonstration in the paper of what the $\pi(y)$ weighting means: when the released model puts little probability on the gold answer, a verdict on a sampled answer rarely finds it, and labels do.

7 Limitations

- **Verdicts come from gold answers.** Flipped, missing and complaint-skewed verdicts are simulated; a rater who is systematically wrong about what was right is not, and real deployments (Kreutzer et al. [29]; Gao et al. [15]) find explicit ratings noisier than this.
- **Seeds.** The eight-pass, one-pass and answer-key arms have three seeds everywhere. Because one Laya encoder run takes about twenty minutes on the GPU we could afford, the encoder arms have three seeds on the public menu and JevBench, two on DecisionBench and one on typed-decisions and emotion, and none on AG News; the realistic-feedback and no-KL arms have three seeds on DecisionBench and one elsewhere, and were run on Vega only; the Laya head-only arms were run for one pass (three seeds on typed-decisions, one on the public menu and DecisionBench); the Laya head-only eight-pass arms are taken from the first version's grid, where the labelled head had no KL term. Single-seed intervals carry row and cluster resampling only.
- **Two released models, four vendor benchmarks.** Vega and Laya are both from one developer and neither has a paper; JevBench and DecisionBench are small and recent. Emotion and AG News are the only standard sets, and on AG News the released models are already at 0.93.
- **Benchmark artifacts and contamination.** Four tasks have learnable gold positions or option structures; Vega's shipped adapter was fitted on the typed-decisions pool; the engines' training corpora are undisclosed, so overlap with the public menu cannot be ruled out. We report around these, not without them.
- **Hyper-parameters.** β, the rank, the learning rates and the eight passes were fixed before this grid, β in a pilot that used DecisionBench test rows. The labelled arms were tuned only by early stopping on calibration

labels; the verdict arms not at all.

- **One temperature per benchmark.** On the 43-task public menu this is too coarse, and ECE there is mostly small-sample estimator bias; a per-task temperature fitted the same way is the obvious next step.
- **Same-seed reruns differ.** Across machines a single run moves by up to 3 points; conclusions should rest on the three-seed means and the intervals, not on single cells.

8 Conclusion

The question we set out to answer was how much of a decision model's labelled-data recipe right / wrong feedback recovers. Under repeated on-policy feedback the answer is most of it, because repeated verdicts on the same rows are largely labels in disguise and the update is a reweighted cross-entropy. The more useful answer comes from the logged arms: one verdict per row, collected on the released model's own sampled answers and trained on until convergence, recovers most of what eight on-policy passes do on Vega, and logging only the answers the system actually served costs a little more. Both stay below the labels, by one to five points, and the gap is widest where the released model rarely puts the right answer near the top: with Vega's shipped typed-decisions adapter switched off, labels gain +31.6 points and verdicts +13.0. On typed-decisions and the public menu much of what verdicts deliver is a label prior that a few dozen numbers could learn; on DecisionBench and emotion it is not. On Laya the matched fine-tune on labels wins where the gap is measurable. None of the parts is new; the measurement is, and so is the list of things that made the first version's numbers look better than they were. The next experiment is the one this grid cannot run: a model deployed with real users, their verdicts logged once on the answer it actually gave, and the switch deciding whether the add-on stays.

Acknowledgements. We thank the authors of VegaML and Laya for releasing their models, benchmark code and training recipes, and the reviewers of the first version, whose report is reproduced in the repository.

References

[1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of ACL*, 2024.

[2] Nishant Balepur, Abhilasha Ravichander, and Rachel Rudinger. Artifacts or abduction: how do LLMs answer multiple-choice questions without the question? In *Proceedings of ACL*, 2024.

[3] Jon Ander Campos, Kyunghyun Cho, Arantxa Otegi, Aitor Soroa, Gorka Azkune, and Eneko Agirre. Improving conversational question answering systems after deployment using feedback-weighted learning. In *Proceedings of COLING*, 2020.

[4] Leshem Choshen, Lior Fox, Zohar Aizenbud, and Omri Abend. On the weaknesses of reinforcement learning for neural machine translation. In *Proceedings of ICLR*, 2020.

[5] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, 2017.

[6] Convai Innovations. Laya: fast, non-autoregressive System 1 decision engine with calibrated probabilities. https://huggingface.co/convaiinnovations/laya, 2026. Python package `laya` 0.3.23; checkpoints `convaiinnovations/laya` and `convaiinnovations/laya-typed-decisions`.

[7] Koby Crammer and Claudio Gentile. Multiclass classification with bandit feedback using adaptive regularization. In *Proceedings of ICML*, 2011.

[8] Mehul Damani, Isha Puri, Stewart Slocum, Idan Shenfeld, Leshem Choshen, Yoon Kim, and Jacob Andreas. Beyond binary rewards: training LMs to reason about their uncertainty. In *Proceedings of ICLR*, 2026.

[9] Amit Daniely and Tom Helbertal. The price of bandit information in multiclass online classification. In *Proceedings of COLT*, 2013.

[10] Damek Davis and Benjamin Recht. Reinforcement learning with binary rewards is gradient ascent on a transform of the success probability. *arXiv preprint arXiv:2510.13651*, 2025.

[11] Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: reward ranked finetuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023.

[12] Miroslav Dudík, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *Proceedings of ICML*, 2011.

[13] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: model alignment as prospect theoretic optimization. In *Proceedings of ICML*, 2024.

[14] Ge Gao, Eunsol Choi, and Yoav Artzi. Simulating bandit learning from user feedback for extractive question answering. In *Proceedings of ACL*, 2022.

[15] Ge Gao, Hung-Ting Chen, Yoav Artzi, and Eunsol Choi. Continually improving extractive QA via human feedback. In *Proceedings of EMNLP*, 2023.

[16] Aritra Ghosh, Himanshu Kumar, and P. S. Sastry. Robust loss functions under label noise for deep neural networks. In *Proceedings of AAAI*, 2017.

[17] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[18] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of ICML*, 2017.

[19] Chirag Gupta and Aaditya Ramdas. Top-label calibration and multiclass-to-binary reductions. In *Proceedings of ICLR*, 2022.

[20] Braden Hancock, Antoine Bordes, Pierre-Emmanuel Mazaré, and Jason Weston. Learning from dialogue after deployment: feed yourself, chatbot! In *Proceedings of ACL*, 2019.

[21] Hanno Labs. DecisionBench. https://huggingface.co/datasets/Hanno-Labs/decision-bench, 2026.

[22] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of ICLR*, 2022.

[23] Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, Craig Ferguson, Agata Lapedriza, Noah Jones, Shixiang Gu, and Rosalind Picard. Way off-policy batch deep reinforcement learning of implicit human preferences in dialog. In *arXiv preprint arXiv:1907.00456*, 2019.

[24] Thorsten Joachims, Adith Swaminathan, and Maarten de Rijke. Deep learning with logged bandit feedback. In *Proceedings of ICLR*, 2018.

[25] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Dodds, Andy Jones, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[26] Sham M. Kakade, Shai Shalev-Shwartz, and Ambuj Tewari. Efficient bandit algorithms for online multiclass prediction. In *Proceedings of ICML*, 2008.

[27] Samuel Kiegeland and Julia Kreutzer. Revisiting the weaknesses of reinforcement learning for neural machine translation. In *Proceedings of NAACL*, 2021.

[28] Julia Kreutzer, Artem Sokolov, and Stefan Riezler. Bandit structured prediction for neural sequence-to-sequence learning. In *Proceedings of ACL*, 2017.

[29] Julia Kreutzer, Shahram Khadivi, Evgeny Matusov, and Stefan Riezler. Can neural machine translation be improved with user feedback? In *Proceedings of NAACL (Industry Track)*, 2018.

[30] Julia Kreutzer, Joshua Uyheng, and Stefan Riezler. Reliability and learnability of human bandit feedback for sequence-to-sequence reinforcement learning. In *Proceedings of ACL*, 2018.

[31] Ananya Kumar, Percy Liang, and Tengyu Ma. Verified uncertainty calibration. In *Advances in Neural Information Processing Systems*, 2019.

[32] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *Proceedings of ICLR*, 2022.

[33] Carolin Lawrence and Stefan Riezler. Improving a neural semantic parser by counterfactual learning from human bandit feedback. In *Proceedings of ACL*, 2018.

[34] Zachary C. Lipton, Yu-Xiang Wang, and Alexander J. Smola. Detecting and correcting for label shift with black box predictors. In *Proceedings of ICML*, 2018.

[35] LocalLLaMA. typed-decisions. `https://huggingface.co/datasets/LocalLLaMA/typed-decisions`, 2026.

[36] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning into quantiles. In *Proceedings of AAAI*, 2015.

[37] NandhaKishorM. VegaML: typed decisions from a frozen language model and a small physics engine. `https://github.com/NandhaKishorM/vegaml`, 2026. Python package `vegaml` 0.8.0; models `nandakishorm/vega-08b-public-intents` and the 4B release.

[38] Khanh Nguyen, Hal Daumé III, and Jordan Boyd-Graber. Reinforcement learning for bandit neural machine translation with simulated human feedback. In *Proceedings of EMNLP*, 2017.

[39] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

[40] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[41] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of ICML*, 2021.

[42] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

[43] Rebecca Roelofs, Nicholas Cain, Jonathon Shlens, and Michael C. Mozer. Mitigating bias in calibration error estimation. In *Proceedings of AISTATS*, 2022.

[44] Marco Saerens, Patrice Latinne, and Christine Decaestecker. Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure. *Neural Computation*, 14(1):21–41, 2002.

[45] Elvis Saravia, Hsien-Chi Toby Liu, Yen-Hao Huang, Junlin Wu, and Yi-Shin Chen. CARER: Contextualized affect representations for emotion recognition. In *Proceedings of EMNLP*, 2018.

[46] Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL's razor: why online reinforcement learning forgets less. In *Proceedings of ICLR*, 2026.

[47] Artem Sokolov, Julia Kreutzer, Christopher Lo, and Stefan Riezler. Learning structured predictors from bandit feedback for interactive NLP. In *Proceedings of ACL*, 2016.

[48] Florian Standhartinger. JevBench: public tiers. https://huggingface.co/datasets/fstandhartinger/jevbench, 2026.

[49] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems*, 2020.

[50] Adith Swaminathan and Thorsten Joachims. Counterfactual risk minimization: learning from logged bandit feedback. In *Proceedings of ICML*, 2015.

[51] Yuanzhi Wang and Xinyu Qiao. Bandit-feedback online multiclass classification with conformal prediction. In *Proceedings of ICML*, 2024.

[52] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, et al. Smarter, better, faster, longer: a modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

[53] Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, and Ludwig Schmidt. Robust fine-tuning of zero-shot models. In *Proceedings of CVPR*, 2022.

[54] Wei Xiong, Jiarui Yao, Yuhui Xu, Bo Pang, Lei Wang, Doyen Sahoo, Junnan Li, Nan Jiang, Tong Zhang, Caiming Xiong, and Hanze Dong. A minimalist approach to LLM reasoning: from rejection sampling to REINFORCE. *arXiv preprint arXiv:2504.11343*, 2025.

[55] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, 2025.

[56] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, 2022.

[57] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, 2015.

[58] Zhilu Zhang and Mert R. Sabuncu. Generalized cross entropy loss for training deep neural networks with noisy labels. In *Advances in Neural Information Processing Systems*, 2018.

[59] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: improving few-shot performance of language models. In *Proceedings of ICML*, 2021.

[60] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *Proceedings of ICLR*, 2024.

[61] Han Zhou, Xingchen Wan, Lev Proleev, Diana Mincu, Jilin Chen, Katherine Heller, and Subhrajit Roy. Batch calibration: rethinking calibration for in-context learning and prompt engineering. In *Proceedings of ICLR*, 2024.

[62] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

A What the review found, and what changed

The first version of this paper (repository commit `cd52a9f`) was reviewed by a panel of thirteen automated specialist reviewers with an independent verifier for every major finding; the full report, its per-row scripts and its verdicts are in the repository under `reports/critique.zip`. Every number it quotes reproduced from our records, and it found no test-set leakage and no cherry-picked reruns. Its objections were to what was compared with what, and to what was claimed. Table 12 lists each major finding and what this version does about it.

Table 12: The review's major findings and this version's response.

finding	response
"One bit per row" misdescribed the protocol: eight on-policy passes rate each row up to eight times and reveal the gold answer for most rows.	Every feedback arm now has a one-pass version in which each row is rated once; the one-pass number is reported as the deployment figure. Reveal statistics are measured and reported (Table 6). The "$\log_2 k$ bits against one" argument is gone.
The expected update of REINFORCE with a 0/1 reward is cross-entropy weighted by $\pi(y)$; the gains are a known effect.	Stated in Section 4 and cited; the fix / break analysis by gold rank tests the prediction (Table 7). The paper no longer claims a new learning setting.
The switch compared sampled answers, so it measured expected gold probability, not deployed accuracy; it threw away real gains on 641- and 1,200-row calibration sets and kept a loss on emotion.	The switch compares best answers (two verdicts per row). The old statistic is still computed and shown beside it (Figure 8).
Laya with the encoder unfrozen was compared with a fine-tune that never saw the benchmark; the matched answer-key encoder fine-tune had been dropped.	The matched arm (`sup_enc`: same rows, passes, KL term, encoder unfrozen) is run on every benchmark but AG News, with and without early stopping. `laya-typed-decisions` is presented as in-domain only on typed-decisions.
Laya's answer-key head was trained without the KL term the paper described.	It now has the same KL term as Vega's adapter.
The answer-key adapter was called an upper bound but was untuned and over-trained by epoch 8.	An early-stopped version (epoch chosen on the calibration rows' labels) is reported next to the same-recipe version; neither is called an upper bound.
Win counts counted statistical ties and used six as the denominator where TTT is defined on five benchmarks; the answer-key arms were not run on JevBench.	Counts are by Holm-adjusted significance within a family, out of the benchmarks where the comparison exists; every answer-key arm is cross-fitted on JevBench.
Intervals ignored seed variance; the KL ablation had none.	All intervals resample seeds and row clusters; p-values are Holm-adjusted; the KL contrasts carry intervals.
More than half of the 0.8B public-menu gain came from truthfulqa_mc1 (gold always first) and pubmedqa (majority class); about a quarter of the DecisionBench gain from two artifact tasks.	Disclosed in Section 5; every public and DecisionBench number is reported with and without those tasks (Table 9); per-task tables for both.
A per-task label prior with no features reproduces the typed-decisions gains and part of the others.	The prior baseline is run from verdicts and from labels on every benchmark (Table 6, Figure 5).
Vega's shipped typed_decisions adapter was fitted on the typed-decisions feedback pool.	Disclosed; typed-decisions on Vega is also run with the shipped adapters off (Table 11).
The feedback pools were class-balanced against naturally distributed test sets (emotion, public).	Emotion and AG News pools are now random draws with the natural class mix; the public pool is VegaML's protocol and is described as such.
The ECE comparison with the base model credited temperature scaling to the adapter; calibration was reported as ECE only.	The released model with the same verdict-fitted temperature (and a label-fitted one) is a reported control; NLL, Brier and AURC appear beside ECE (Table 10).
TTT was scored under vegaml 0.8.0's rule; 0.12.3 requires the head to beat the engine as well.	Both rules are reported.
The honesty fix is standard top-label temperature scaling from correctness bits; over-sharpening after policy gradient is known.	Positioned as such in Sections 4 and Related work.
The no-engine control used an entropy bonus, not the KL term the paper said, and sat at chance.	Dropped from the paper.
β was chosen on DecisionBench test rows; $\beta = 1$ runs were unreported; the KL pilot did not fully reproduce.	Disclosed in Section 4 and below.
The reward curve was presented as a bound on live exploration cost; passes 2–8 are in-sample.	Replaced by the pass-1 reward against the released model's best-answer accuracy on the same rows (Table 6).
Many prose numbers were hard-coded and several contradicted the tables.	Every number in the text is a macro generated from the records.
The encoder-unfrozen variant was added after the grid was called final.	Every arm reported from this version's grid was run in one grid; the only numbers quoted from the first version's grid are the Laya head-only eight-pass arms, marked as such.

The β pilot. $\beta = 0.1$ was chosen in October 2026 from $\{0, 0.1, 1\}$ in a pilot on DecisionBench that trained on 645 and tested on 430 of the README's 1,075 test rows, and in a learning-curve study whose test set overlapped the README slice. In that pilot $\beta = 1$ kept about 80% of the gain at 0.8B; in the learning-curve study it kept about a third. The first version's grid also ran $\beta = 1$ on the public menu and JevBench at 0.8B without reporting it: $\beta = 1$ scored 0.584 against 0.590 for $\beta = 0.1$ on the public menu and 0.548 against 0.517 on JevBench (released model 0.550). An independent reproduction of the first version's DecisionBench study on another machine recovered the headline gain (+10.4 against +10.2) but not the KL pilot's gap ($\beta = 0$: 0.566 against 0.571 for $\beta = 0.1$, where the pilot had 0.536 against 0.567). None of this was retuned for this version; $\beta = 0.1$ is used throughout and the no-KL ablation has intervals.

B Reproducing the results

`experiments/peekaboo_remaining/run.py --gridv2` runs the whole grid: it reads every benchmark once through every model and caches the backbone features and the official records, replays each released model against its official prediction call, then runs every method and seed and writes `output_v2/TABLES.md`. Finished steps are skipped, so the run can be resumed. `analysis.py` computes every interval, control and statistic in this paper from the per-row records, and `paper/make_figures.py` produces every table, figure and number macro. The first version's grid (`--grid v1`) and its results under `output/` are kept for comparison; its DecisionBench learning-curve study is run by `experiments/vega_policy/reproduce.py`.

Checks. Replaying each released model on its cached inputs matches the official prediction call to 0 for Vega and to 5×10^{-5} for Laya (one question per forward pass, as Laya's own call runs it). The released models reproduce their README figures: Vega 0.8B and 4B on the public menu at 0.556 and 0.718 against the README's 0.556 and 0.717, DecisionBench at 0.509 and 0.636 against 0.509 and 0.633; Laya and laya-typed-decisions on typed-decisions at 0.362 and 0.761 against 0.362 and 0.766. Results on the same seed differ across machines by up to 3 points for a single run (floating-point non-determinism in feature extraction and training kernels); the three-seed means are what reproduce.

C Per-task results

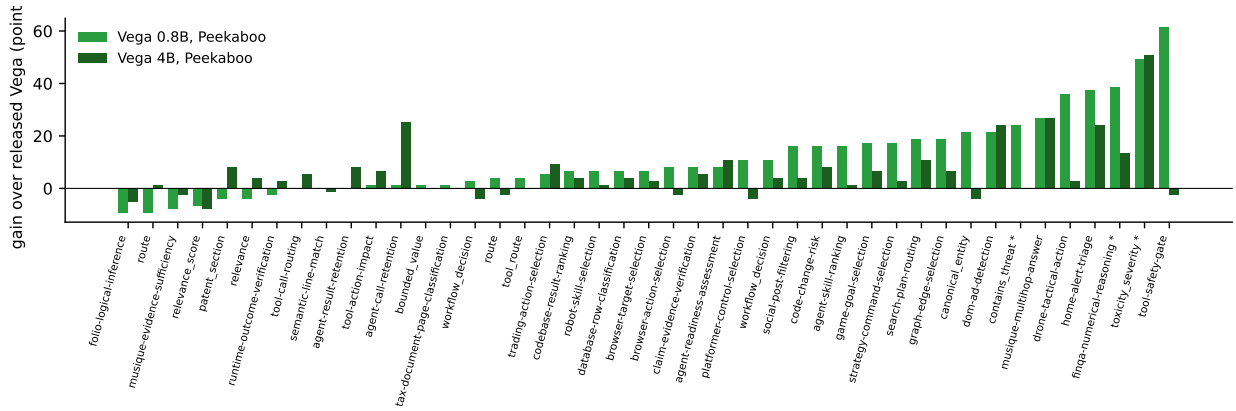


Figure 10: DecisionBench, gain of eight-pass Peekaboo over released Vega on each of the 43 tasks (25 test rows each), sorted by the 0.8B gain. *: tasks with a construction artifact.

Table 13: Accuracy on each of the 43 public tasks (mean over seeds). key: the early-stopped answer-key adapter / head; Pk8, Pk1: Peekaboo with eight passes, one pass; f-t: laya-typed-decisions; -enc: encoder unfrozen. * marks the two tasks with a construction artifact. Bold: the best method for that model on that task.

task	Vega 0.8B					Vega 4B					Laya			
	base	TTT	key	Pk8	Pk1	base	TTT	key	Pk8	Pk1	base	f-t	key-enc	Pk-enc
aegis_prompt_safety	0.50	0.50	0.64	**0.64**	0.57	0.60	0.60	**0.79**	0.74	0.73	0.37	0.43	0.68	**0.70**
ag_news	**0.97**	**0.97**	**0.97**	**0.97**	**0.97**	0.95	0.95	**0.97**	0.95	0.97	**0.98**	**0.98**	0.97	0.97
anli_r3	0.28	0.28	**0.39**	0.37	0.32	0.35	**0.52**	0.43	0.40	0.39	**0.43**	0.40	0.42	0.42
arc_challenge	0.48	0.48	0.44	**0.50**	0.48	0.80	0.80	0.80	0.81	**0.83**	0.28	**0.37**	0.34	**0.37**
arc_easy	**0.60**	**0.60**	0.55	0.55	0.58	0.92	0.92	**0.95**	0.94	0.93	0.52	0.57	**0.57**	0.57
banking77	0.68	0.68	**0.72**	0.66	0.62	0.73	0.73	**0.74**	0.69	0.69	0.42	0.40	**0.48**	0.37
boolq	0.75	0.75	0.76	**0.82**	0.79	0.82	0.82	0.84	0.84	**0.91**	**0.87**	0.85	0.78	0.83
casehold	0.35	0.35	**0.49**	0.46	0.42	0.55	0.55	**0.63**	0.61	0.63	0.38	0.38	0.38	**0.45**
climate_fever	0.35	0.38	**0.47**	0.38	0.36	0.45	0.38	**0.48**	0.46	**0.48**	0.33	**0.35**	0.25	0.30
clinc150_oos	**0.68**	**0.68**	0.62	0.62	0.61	**0.92**	**0.92**	0.83	0.78	0.83	0.00	**0.32**	0.00	0.00
commonsenseqa	0.35	0.35	**0.41**	0.38	0.35	0.53	0.53	0.59	0.59	**0.60**	0.45	0.45	**0.47**	0.45
cruxeval_output	0.65	0.65	**0.87**	0.79	0.82	0.85	0.92	**0.94**	0.92	**0.94**	0.78	0.73	0.79	**0.81**
dbpedia14	0.60	**0.85**	0.77	0.67	0.72	0.90	**0.97**	0.92	0.93	0.90	0.85	0.83	0.91	**0.93**
emotion	**0.55**	**0.55**	0.43	0.53	0.54	**0.63**	0.45	0.52	0.52	0.61	**0.57**	**0.57**	0.53	0.53
enron_spam	**1.00**	**1.00**	**1.00**	0.99	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	0.99	0.92	0.90	**0.93**	0.92
glue_rte	**0.62**	**0.62**	0.61	**0.62**	0.58	0.87	0.85	0.87	0.86	**0.89**	**0.83**	0.82	0.82	0.80
hellaswag	0.25	0.25	0.27	**0.32**	0.28	**0.65**	**0.65**	0.58	0.53	0.59	0.37	0.37	0.39	**0.39**
jailbreak_cls	**0.98**	**0.98**	**0.98**	0.97	0.96	**1.00**	**1.00**	0.99	0.98	0.99	0.98	0.98	0.98	**0.99**
ledgar	0.33	0.33	**0.59**	0.40	0.39	0.63	0.63	0.63	0.63	**0.66**	0.25	0.18	**0.47**	0.33
massive_intent_en	0.38	0.38	**0.53**	0.43	0.43	0.60	0.60	**0.64**	0.59	0.60	0.38	0.42	**0.47**	0.38
medmcqa	0.18	0.18	0.19	0.28	**0.30**	0.48	0.48	**0.49**	**0.49**	0.48	**0.35**	**0.35**	0.32	0.31
medqa_usmle	0.31	0.31	0.27	0.29	**0.31**	0.51	0.51	**0.55**	0.53	**0.55**	0.20	**0.29**	0.23	0.28
mmlu	**0.37**	**0.37**	0.32	0.32	0.31	**0.73**	**0.73**	0.63	0.57	0.61	0.25	**0.35**	0.31	0.32
mmlu_pro	0.20	0.13	0.16	0.21	**0.24**	**0.40**	**0.40**	0.39	**0.40**	0.38	**0.18**	**0.18**	**0.18**	0.18
openbookqa	**0.58**	**0.58**	0.53	0.54	0.58	**0.83**	**0.83**	0.79	0.78	0.79	0.40	0.48	**0.53**	0.49
prompt_injections	0.75	**0.96**	0.87	0.85	0.76	0.85	**0.96**	0.82	0.89	0.90	0.65	0.60	0.81	**0.84**
pubmedqa*	0.13	**0.58**	0.54	0.55	0.54	0.40	**0.62**	0.60	0.56	0.55	0.53	0.55	**0.64**	0.61
race_middle	0.60	0.60	0.54	**0.61**	0.57	0.82	**0.90**	0.82	0.81	0.82	0.58	0.52	**0.59**	0.53
sciq	0.73	0.73	0.67	0.72	**0.76**	0.92	0.92	**0.95**	0.94	0.93	0.52	0.55	**0.56**	0.52
superglue_cb	0.67	0.69	0.68	**0.75**	0.70	0.82	**0.87**	0.81	0.78	0.82	**0.87**	0.84	0.83	0.79
toxic_chat_jailbreak	**0.98**	**0.98**	0.96	0.91	0.92	0.92	0.92	**0.95**	**0.95**	0.94	0.83	**0.90**	0.84	0.87
toxic_chat_toxicity	**0.95**	0.93	0.93	0.90	0.92	**1.00**	0.98	0.98	0.98	0.98	**0.97**	**0.97**	0.94	0.92
trec_coarse	0.53	0.68	**0.72**	0.61	0.60	0.77	**0.88**	0.83	0.77	0.75	0.57	0.52	**0.89**	0.56
truthfulqa_mc1*	0.34	0.34	**0.77**	0.71	0.53	0.64	0.64	**0.73**	0.72	0.65	0.16	0.24	**0.41**	0.38
tweet_hate	**0.65**	0.63	0.62	0.61	0.53	**0.68**	0.60	0.57	0.61	0.59	**0.73**	0.72	0.64	0.67
tweet_irony	**0.78**	0.60	0.71	0.67	0.70	**0.78**	**0.78**	0.74	0.73	0.71	**0.78**	0.74	0.71	0.73
tweet_offensive	0.80	0.80	0.81	0.80	**0.81**	0.78	0.78	**0.85**	0.78	0.80	0.69	0.71	**0.79**	0.78
tweet_sentiment	0.44	0.44	0.45	**0.50**	0.46	**0.58**	**0.58**	0.49	0.56	0.49	**0.56**	0.49	0.45	0.45
tweet_stance_abortion	0.36	0.45	0.48	**0.58**	0.40	**0.55**	0.49	0.48	0.31	0.45	0.31	0.31	0.38	**0.44**
unfair_tos	**0.74**	0.65	0.56	0.57	0.54	**0.88**	0.70	0.82	0.73	0.71	**0.88**	0.84	0.61	0.61
vitaminc	0.47	0.47	**0.54**	0.47	0.53	0.62	0.62	0.63	0.61	**0.64**	0.43	0.33	**0.66**	0.62
winogrande	0.50	0.50	**0.52**	0.49	0.51	0.58	0.58	0.62	**0.63**	**0.63**	0.53	0.50	0.53	**0.57**
yelp_stars	0.50	0.50	**0.59**	0.49	0.45	**0.58**	**0.58**	0.52	0.47	0.52	0.28	0.37	**0.59**	0.38

Table 14: DecisionBench per task (25 test rows each): accuracy of the released model and gain in points of Peekaboo (8 passes), the answer-key adapter and TTT (0.8.0 rule). *: tasks with a construction artifact.

task	Vega 0.8B				Vega 4B			
	base	Pk	key	TTT	base	Pk	key	TTT
tool-action-impact	0.68	+1	+1	+4	0.60	+7	+15	+12
agent-skill-ranking	0.40	+16	+8	+0	0.64	+1	+0	+0
tool-call-routing	0.52	+0	-5	+0	0.52	+5	+0	+0
tool-safety-gate	0.32	+61	+64	+60	0.96	-3	+4	+0
runtime-outcome-verification	0.48	-3	-3	+4	0.44	+3	+0	+8
browser-action-selection	0.52	+8	+16	+0	0.76	-3	-9	+0
browser-target-selection	0.12	+7	+5	+0	0.32	+3	-5	+0
codebase-result-ranking	0.48	+7	-3	+0	0.52	+4	-7	+0
semantic-line-match	0.96	+0	-5	+0	0.96	-1	-5	-4
agent-readiness-assessment	0.72	+8	+12	+0	0.80	+11	+11	+0
agent-call-retention	0.92	+1	-4	-4	0.68	+25	+32	+28
agent-result-retention	1.00	+0	+0	+0	0.92	+8	+8	+0
database-row-classification	0.44	+7	-5	+0	0.40	+4	-3	+0
relevance_score	0.36	-7	+5	+8	0.52	-8	-8	-4
claim-evidence-verification	0.52	+8	+5	-4	0.80	+5	+0	-12
trading-action-selection	0.40	+5	-3	+0	0.36	+9	+9	+0
finqa-numerical-reasoning*	0.16	+39	+49	+0	0.44	+13	+21	+0
route	0.40	+4	+8	+0	0.76	-3	-7	+0
game-goal-selection	0.40	+17	+11	+0	0.52	+7	+12	+0
platformer-control-selection	0.16	+11	+5	+4	0.36	-4	+8	+4
strategy-command-selection	0.16	+17	+19	+0	0.32	+3	+8	+0
route	0.76	-9	+0	+0	0.80	+1	+0	+0
home-alert-triage	0.44	+37	+40	+0	0.64	+24	+23	+0
graph-edge-selection	0.32	+19	+20	+0	0.80	+7	-4	+0
workflow_decision	0.48	+11	+8	+20	0.84	+4	-1	+8
workflow_decision	0.72	+3	+3	+4	0.88	-4	-4	-12
folio-logical-inference	0.44	-9	-9	-24	0.60	-5	-9	-24
musique-evidence-sufficiency	0.60	-8	-1	-24	0.60	-3	-7	-24
musique-multihop-answer	0.52	+27	+33	+0	0.40	+27	+33	+0
contains_threat*	0.76	+24	+24	+0	1.00	+0	+0	+0
toxicity_severity*	0.20	+49	+48	+48	0.20	+51	+49	+24
bounded_value	0.88	+1	+4	+0	1.00	+0	-3	+0
canonical_entity	0.72	+21	+21	+0	0.96	-4	-5	+0
drone-tactical-action	0.16	+36	+52	+0	0.64	+3	+11	+0
robot-skill-selection	0.40	+7	+13	+0	0.48	+1	-7	+0
search-plan-routing	0.56	+19	+15	+0	0.72	+11	+4	+0
social-post-filtering	0.56	+16	+16	+12	0.84	+4	+1	-12
tool_route	0.84	+4	+1	+0	0.92	+0	+0	+0
code-change-risk	0.44	+16	+21	+0	0.52	+8	+12	-4
tax-document-page-classification	0.04	+1	+12	+0	0.12	+0	+3	+0
patent_section	0.40	-4	-1	-12	0.28	+8	+16	+4
relevance	0.76	-4	+4	-12	0.76	+4	+0	+8
dom-ad-detection	0.76	+21	+24	+8	0.76	+24	+24	+16

D Hyper-parameters

add-on, Vega	gated LoRA, rank 8, on seven engine layers (`vegaml.attach_adapter`); 409k / 857k parameters
add-on, Laya	trainable copy of type embedding, 2-layer decision head and scorer (26.5M); encoder frozen, or a trained copy ("encoder unfrozen", 421M in all)
optimiser	Adam; lr 10^{-3} (Vega adapter), 10^{-4} (Laya head), 2.5×10^{-5} (Laya encoder); grad-norm clip 1.0
schedule	8 passes (or 1), batches of 32 (encoder: micro-batches of 8, one update per 32 rows)
reward	$r = \mathbf{1}[\text{sampled answer} = \text{gold}]$; baseline $b \leftarrow 0.9\,b + 0.1\,\bar{r}$, $b_0 = 0.5$
KL term	$\beta = 0.1$ for every trained arm (ablation: 0)
answer-key arms	cross-entropy on the gold answer, same KL term; early stopping picks the epoch with the best argmax accuracy on the calibration rows
logged arms	best answer: $-r \log \pi(a) - (1 - r) \log(1 - \pi(a))$ on the released model's argmax, rated once; sampled: clipped importance weights (cap 10) on one answer sampled from the released model
honesty fix	T on a 61-point log grid in $[0.25, 32]$, maximising the Bernoulli likelihood of verdicts on sampled answers from the calibration rows; label arms: NLL temperature scaling on the same rows
switch	keep the add-on if its best answers are right more often than the released model's on the calibration rows
label prior	one log-offset per (task, option text); same loop, lr from $\{10^{-3}, 10^{-2}, 3 \times 10^{-2}, 10^{-1}\}$ chosen on the calibration rows' verdicts
splits	feedback rows 80 / 20 train / calibration per task by a fixed permutation; JevBench 5-fold by paraphrase group, calibration fifth by paraphrase group
intervals	2,000 bootstrap replicates resampling seeds and row clusters; Holm within families; non-inferiority margin 2 points
seeds	0, 1, 2